

Modèle pour l'Ingénierie des Systèmes

— UML —

Cédric Buche

École Nationale d'Ingénieurs de Brest (ENIB)

28 janvier 2014



1 Diagramme d'états-transitions

2 Diagramme d'activités

3 Génération de code

Définitions — rôles

Vue dynamique : évolutions possibles des états d'un classifieur

- Objectifs : décrire le **comportement** interne d'un classifieur
 - Ensemble des états possibles (pertinents)
 - Possibilités de changements d'état sur occurrence d'événement
- Origine : *Statecharts* (David HAREL, 1987)
- **Machine à états** :
le comportement ne dépend pas uniquement des entrées du système, mais aussi de son « histoire », son état interne
- Type de modèle : **système discret** :
 - comportement caractérisable (= observable) par une succession d'états
 - avec changement d'état discontinu (= *discret*)

Définitions — rôles

Vue dynamique : évolutions possibles des états d'un classifieur

- Objectifs : décrire le **comportement** interne d'un classifieur
 - Ensemble des états possibles (pertinents)
 - Possibilités de changements d'état sur occurrence d'événement
- Origine : *Statecharts* (David HAREL, 1987)
- **Machine à états** :
le comportement ne dépend pas uniquement des entrées du système, mais aussi de son « histoire », son état interne
- Type de modèle : **système discret** :
 - comportement caractérisable (= observable) par une succession d'états
 - avec changement d'état discontinu (= *discret*)

Définitions — rôles

Vue dynamique : évolutions possibles des états d'un classifieur

- Objectifs : décrire le **comportement** interne d'un classifieur
 - Ensemble des états possibles (pertinents)
 - Possibilités de changements d'état sur occurrence d'événement
- Origine : *Statecharts* (David HAREL, 1987)
- **Machine à états** :
le comportement ne dépend pas uniquement des entrées du système, mais aussi de son « histoire », son état interne
- Type de modèle : **système discret** :
 - comportement caractérisable (= observable) par une succession d'états
 - avec changement d'état discontinu (= *discret*)

Définitions — rôles

Vue dynamique : évolutions possibles des états d'un classifieur

- Objectifs : décrire le **comportement** interne d'un classifieur
 - Ensemble des états possibles (pertinents)
 - Possibilités de changements d'état sur occurrence d'événement
- Origine : *Statecharts* (David HAREL, 1987)
- **Machine à états** :
le comportement ne dépend pas uniquement des entrées du système, mais aussi de son « histoire », son état interne
- Type de modèle : **système discret** :
 - comportement caractérisable (= observable) par une succession d'états
 - avec changement d'état discontinu (= *discret*)

Notion et exemple d'automate à états finis

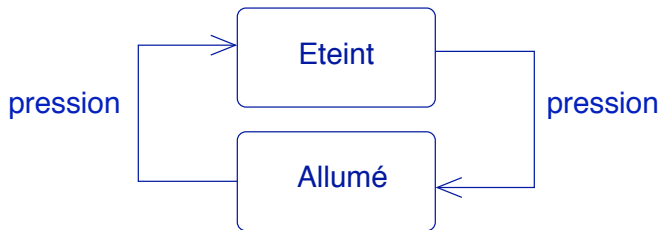
Exemple : télérupteur dans une maison

- **Description du comportement** : Lorsque l'on appuie sur un bouton poussoir, la réaction de l'éclairage associé dépend de son état courant (donc de son historique) :
 - si la lumière est allumée, elle s'éteint,
 - si elle est éteinte, elle s'allume.
- **Éléments du comportement du système** :
 - 2 états : *Allumé* et *Éteint*
 - 2 transitions : $Allumé \mapsto Éteint$ et $Éteint \mapsto Allumé$
 - 1 événement : *pression* sur un bouton poussoir

Notion et exemple d'automate à états finis

Exemple : télérupteur dans une maison

- **Description du comportement** : Lorsque l'on appuie sur un bouton poussoir, la réaction de l'éclairage associé dépend de son état courant (donc de son historique) :
 - si la lumière est allumée, elle s'éteint,
 - si elle est éteinte, elle s'allume.
- **Éléments du comportement du système** :
 - 2 états : *Allumé* et *Éteint*
 - 2 transitions : $Allumé \mapsto Éteint$ et $Éteint \mapsto Allumé$
 - 1 événement : *pression* sur un bouton poussoir



Deux types de modélisation

Deux interprétations du modèle

① Machine à état *comportementale* :

- *behavioral statemachine* : **stm** {behavior}
- spécifier le comportement d'un classifieur, instance d'une classe, cas d'utilisation, collaboration, méthode
- **ce que fait un classifieur** quand on le sollicite
- **sémantique opérationnelle** : implémentable pour contrôler le classifieur

② Machine à état de *protocole* :

- *protocol statemachine* : **stm** {protocol}
- spécifier les séquencements « légaux » de sollicitations
- **comment utiliser un classifieur**
- sans effet sur une instance du classifieur

Deux types de modélisation

Deux interprétations du modèle

① Machine à état *comportementale* :

- *behavioral statemachine* : **stm** {behavior}
- spécifier le comportement d'un classifieur, instance d'une classe, cas d'utilisation, collaboration, méthode
- **ce que fait un classifieur** quand on le sollicite
- **sémantique opérationnelle** : implémentable pour contrôler le classifieur

② Machine à état de *protocole* :

- *protocol statemachine* : **stm** {protocol}
- spécifier les séquencements « légaux » de sollicitations
- **comment utiliser un classifieur**
- sans effet sur une instance du classifieur

Concepts de base

- **Principes généraux** des modèles états – transition
 - État
 - Transition
 - Événement
 - **Spécificités du modèle UML**
 - Signal
 - Transition gardée
 - Effet : action – activités
- machine à état comportementale

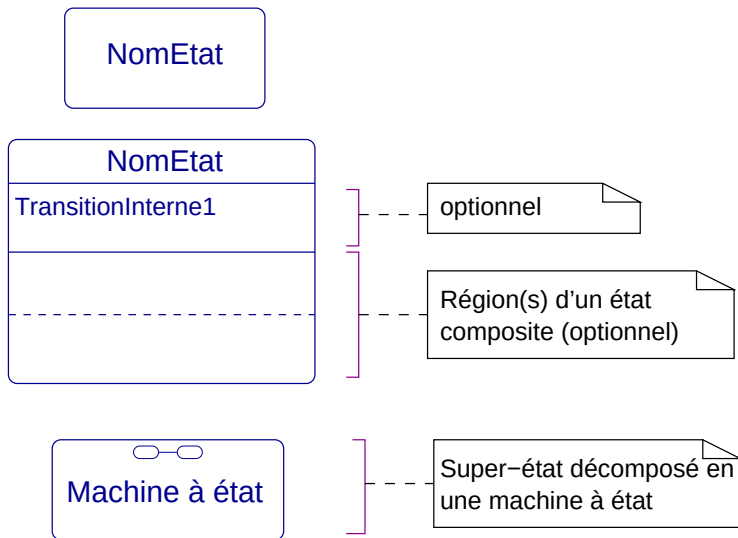
Concepts de base

- **Principes généraux** des modèles états – transition
 - État
 - Transition
 - Événement
 - **Spécificités du modèle UML**
 - Signal
 - Transition gardée
 - Effet : action – activités
- machine à état comportementale

État — définitions

- État simple (sens strict) :
 - potentiellement observable au cours de la vie d'un classifier
 - durée d'activation non nulle
 - défini par un invariant
 - les valeurs d'un ensemble de propriétés du classifier
 - une situation d'attente
 - l'exécution d'un comportement
- Pseudo-état :
 - non observable, généralement de durée nulle
 - rôle : contrôle des activations des états
- État composite
 - peut contenir (envelopper) des sous-états
- État hiérarchique (voir concepts avancés)

État — notations



État initial et état final

Définitions – particularités

- **Pseudo-états**

- Caractérisent l'activation (initiale ou terminale) d'une *région* d'un état englobant

- **État initial** (durée nulle) :

- Indique l'activation initiale de la région englobante en ciblant le premier état actif par défaut $\Rightarrow$ 1 seul par région
- Transition : *restrictions*
 - non déclenchée par un événement,
 - éventuellement gardée (sous réserve...),
 - éventuellement associée à un *effet*

- **État final** :

- Indique que l'activation de la région englobante est terminée
- Aucun autre état n'est alors activable
- Transition : pas de restriction

État initial et état final

Définitions – particularités

- **Pseudo-états**

- Caractérisent l'activation (initiale ou terminale) d'une *région* d'un état englobant

- **État initial** (durée nulle) :

- Indique l'activation initiale de la région englobante en ciblant le premier état actif par défaut $\Rightarrow$ 1 seul par région
- Transition : **restrictions**
 - non déclenchée par un événement,
 - éventuellement gardée (sous réserve...),
 - éventuellement associée à un *effet*

- **État final** :

- Indique que l'activation de la région englobante est terminée
- Aucun autre état n'est alors activable
- Transition : pas de restriction

État initial et état final

Définitions – particularités

- **Pseudo-états**

- Caractérisent l'activation (initiale ou terminale) d'une *région* d'un état englobant

- **État initial** (durée nulle) :

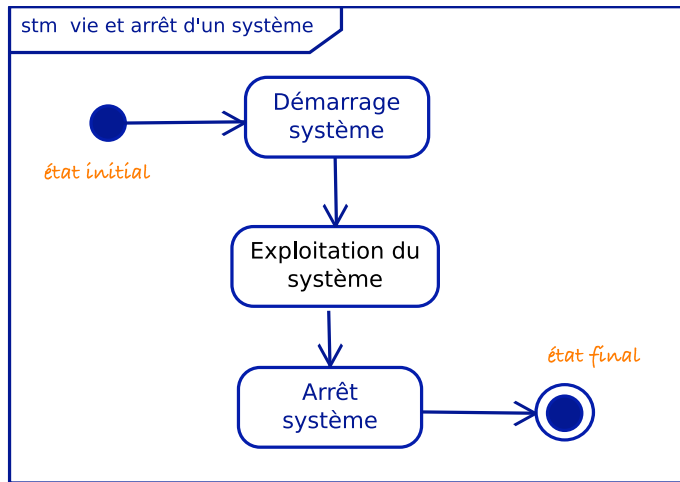
- Indique l'activation initiale de la région englobante en ciblant le premier état actif par défaut $\Rightarrow$ 1 seul par région
- Transition : **restrictions**
 - non déclenchée par un événement,
 - éventuellement gardée (sous réserve...),
 - éventuellement associée à un *effet*

- **État final** :

- Indique que l'activation de la région englobante est terminée
- Aucun autre état n'est alors activable
- Transition : pas de restriction

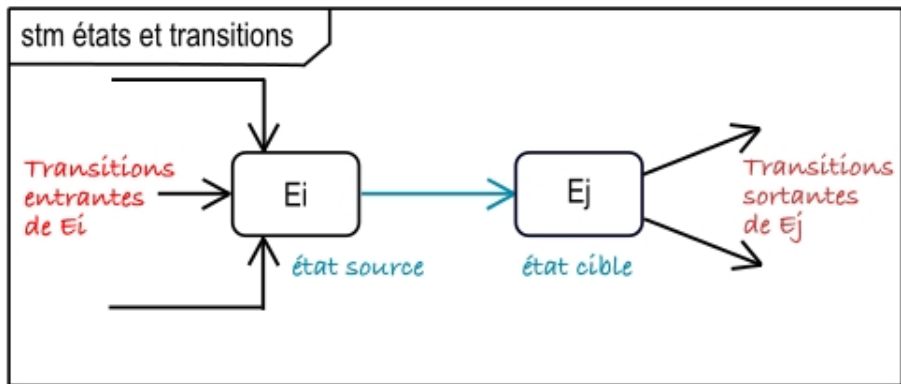
État initial et état final

Notation et exemple d'utilisation



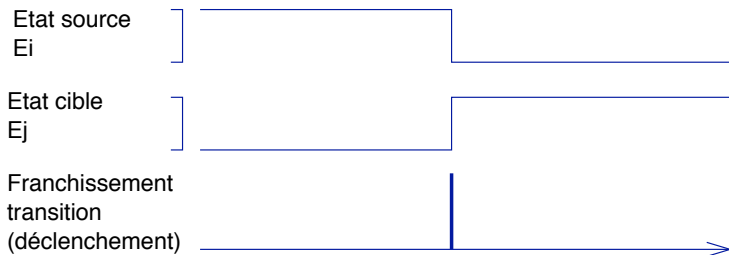
Transition — définition

- Indique un changement possible d'état du classier
- Une transition relie un **état source** à un **état cible**



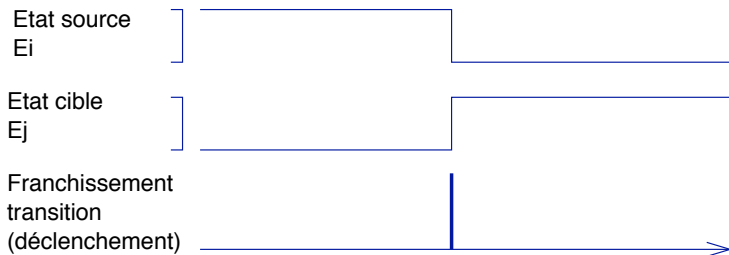
Transition — déclenchement

- Le changement d'état est **instantané** (= atomique)
- Le franchissement (= déclenchement) d'une transition a lieu sur **occurrence d'un événement**
- il peut aussi dépendre d'une **condition de garde**
- Le franchissement peut avoir un **effet** sur le classier
= réalisation d'une action



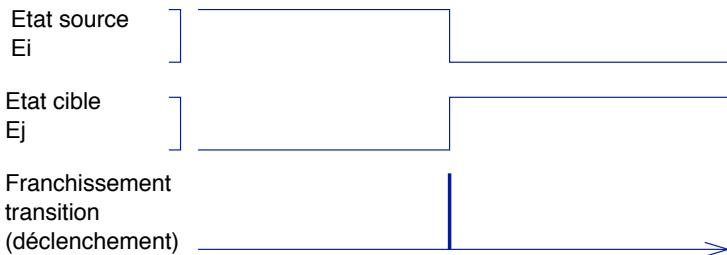
Transition — déclenchement

- Le changement d'état est **instantané** (= atomique)
- Le franchissement (= déclenchement) d'une transition a lieu sur **occurrence d'un événement**
- il peut aussi dépendre d'une **condition de garde**
- Le franchissement peut avoir un **effet** sur le classier
= réalisation d'une action



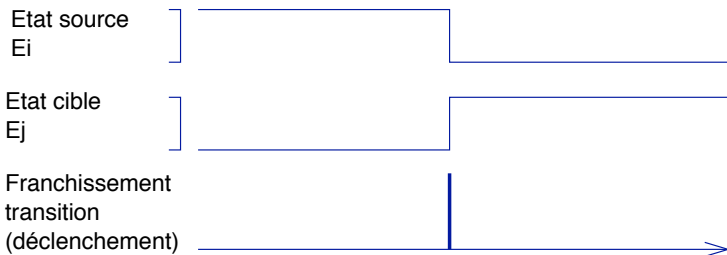
Transition — déclenchement

- Le changement d'état est **instantané** (= atomique)
- Le franchissement (= déclenchement) d'une transition a lieu sur **occurrence d'un événement**
- il peut aussi dépendre d'une **condition de garde**
- Le franchissement peut avoir un **effet** sur le classier
= réalisation d'une action



Transition — déclenchement

- Le changement d'état est **instantané** (= atomique)
- Le franchissement (= déclenchement) d'une transition a lieu sur **occurrence d'un événement**
- il peut aussi dépendre d'une **condition de garde**
- Le franchissement peut avoir un **effet** sur le classier
= réalisation d'une action



Événement — définition

- Quelque chose qui se produit *à un instant donné* lors de l'exécution
- Un événement peut véhiculer une information (paramètres) entre un objet émetteur et un objet récepteur
- Un événement est associé à une transition (ou plusieurs)
- **Occurrence d'un événement** :
 - instance d'un type d'événement
 - instantané (durée nulle)
 - peut provoquer le déclenchement d'une transition (éventuellement plusieurs)



Événement — types prédéfinis

- *call event* : `nomOppe(parameter: type, ...)`
 - réception d'un appel d'invocation d'une opération
 - le déclenchement de la transition provoque l'exécution de l'opération
 - l'émetteur retrouve le contrôle lorsque l'opération est exécutée
- *change event* : `when(<expression>)`
 - événement occurrent quand <expression> devient vraie
 - pas de paramètre
- *signal event* : `nomSignal(parameter: type, ...)`
 - communication asynchrone entre objets
 - (voir la suite)
- *time event* : `after(<time>)`
 - <time> : expression temporelle spécifiant un instant ou durée

Événement — types prédéfinis

- *call event* : `nomOppe(parameter: type, ...)`
 - réception d'un appel d'invocation d'une opération
 - le déclenchement de la transition provoque l'exécution de l'opération
 - l'émetteur retrouve le contrôle lorsque l'opération est exécutée
- *change event* : `when(<expression>)`
 - événement occurrent quand <expression> devient vraie
 - pas de paramètre
- *signal event* : `nomSignal(parameter: type, ...)`
 - communication asynchrone entre objets
 - (voir la suite)
- *time event* : `after(<time>)`
 - <time> : expression temporelle spécifiant un instant ou durée

Événement — types prédéfinis

- *call event* : `nomOppe(parameter: type, ...)`
 - réception d'un appel d'invocation d'une opération
 - le déclenchement de la transition provoque l'exécution de l'opération
 - l'émetteur retrouve le contrôle lorsque l'opération est exécutée
- *change event* : `when(<expression>)`
 - événement occurrent quand <expression> devient vraie
 - pas de paramètre
- *signal event* : `nomSignal(parameter: type, ...)`
 - communication asynchrone entre objets
 - (voir la suite)
- *time event* : `after(<time>)`
 - <time> : expression temporelle spécifiant un instant ou durée

Événement — types prédéfinis

- *call event* : `nomOppe(parameter: type, ...)`
 - réception d'un appel d'invocation d'une opération
 - le déclenchement de la transition provoque l'exécution de l'opération
 - l'émetteur retrouve le contrôle lorsque l'opération est exécutée
- *change event* : `when(<expression>)`
 - événement occurrent quand <expression> devient vraie
 - pas de paramètre
- *signal event* : `nomSignal(parameter: type, ...)`
 - communication asynchrone entre objets
 - (voir la suite)
- *time event* : `after(<time>)`
 - <time> : expression temporelle spécifiant un instant ou durée

Événement de type Signal

- Stéréotype de *classifier*
- Un «signal» peut avoir des attributs et éventuellement des opérations (p. ex. pour sa création)
- Un «signal» peut être défini comme une spécialisation d'un autre; la réception d'un *signal event* provoque le déclenchement d'une transition étiquetée par une de ses généralisations
- Sert à la **communication asynchrone explicite** entre objets
- Produit par une action **send**
- Peut être destiné à un ou plusieurs objets (explicites)

Transition d'achèvement

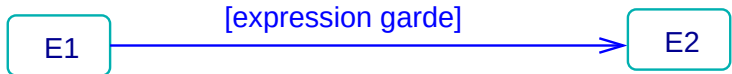
- Une transition dépourvue d'événement déclencheur explicite
- Elle se déclenche à la fin de l'exécution de l'activité de l'état source (y compris les états imbriqués)
- Elle peut avoir une condition de garde
Celle-ci est évaluée quand l'activité de l'état source s'achève ; elle ne l'est plus ensuite

Transition d'achèvement

- Une transition dépourvue d'événement déclencheur explicite
- Elle se déclenche à la fin de l'exécution de l'activité de l'état source (y compris les états imbriqués)
- Elle peut avoir une condition de garde
Celle-ci est évaluée quand l'activité de l'état source s'achève; elle ne l'est plus ensuite

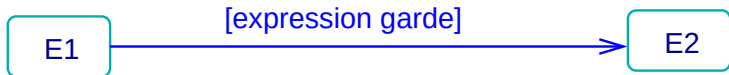
Condition de garde d'une transition

- expression booléenne évaluée dans le contexte de l'objet (classifieur) détenteur de la machine à état
- expression logique sur les attributs de l'objet, ainsi que sur les paramètres de l'événement déclencheur
- évaluée que si la transition est éligible à son déclenchement
- en pratique, peut correspondre à une opération ayant un paramètre de retour de type boolean ; l'opération doit être une requête ({query})



Condition de garde d'une transition

- expression booléenne évaluée dans le contexte de l'objet (classifieur) détenteur de la machine à état
- expression logique sur les attributs de l'objet, ainsi que sur les paramètres de l'événement déclencheur
- évaluée que si la transition est éligible à son déclenchement
- en pratique, peut correspondre à une opération ayant un paramètre de retour de type boolean; l'opération doit être une requête ({query})



Effet (action ou activité) associé à une transition

Machine à états comportementale

- Comportement exécuté lors du franchissement d'une transition
- peut modifier l'objet détenteur de la machine à état
- peut utiliser les paramètres de l'événement déclencheur (événement en cours)
- *sémantique run-to-completion* $\implies$ aucun nouvel événement n'est traité pendant son exécution
- plusieurs actions peuvent être associées à la même transition
- en pratique : se limiter à des actions « courtes » (voir transitions internes pour les actions « étendues »)



Effet (action ou activité) associé à une transition

Machine à états comportementale

- Comportement exécuté lors du franchissement d'une transition
- peut modifier l'objet détenteur de la machine à état
- peut utiliser les paramètres de l'événement déclencheur (événement en cours)
- **sémantique** *run-to-completion* $\implies$ aucun nouvel événement n'est traité pendant son exécution
- plusieurs actions peuvent être associées à la même transition
- en pratique : se limiter à des actions « courtes » (voir transitions internes pour les actions « étendues »)



Effet (action ou activité) associé à une transition

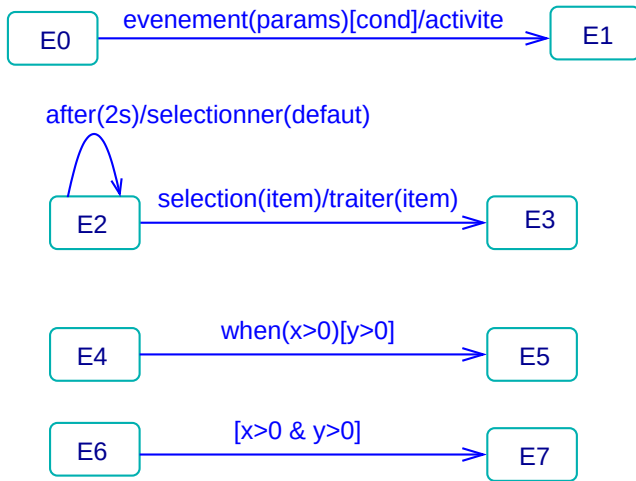
Machine à états comportementale

- Comportement exécuté lors du franchissement d'une transition
- peut modifier l'objet détenteur de la machine à état
- peut utiliser les paramètres de l'événement déclencheur (événement en cours)
- **sémantique** *run-to-completion* $\implies$ aucun nouvel événement n'est traité pendant son exécution
- plusieurs actions peuvent être associées à la même transition
- en pratique : se limiter à des actions « courtes » (voir transitions internes pour les actions « étendues »)



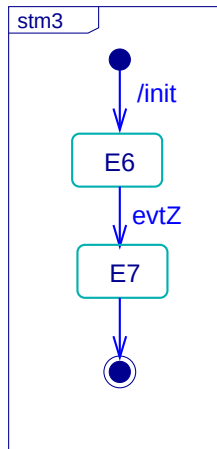
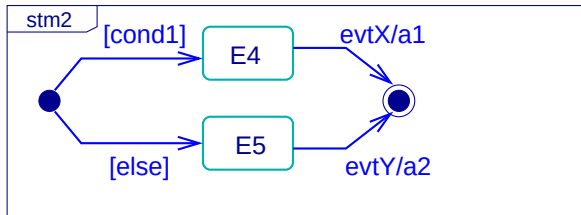
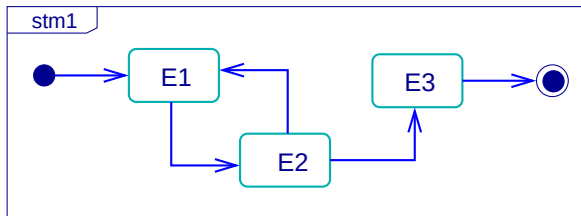
Transition simple : synthèse

Machine à états comportementale



Transitions — État initial et état final

Exemples d'utilisation



Sémantique des machines à état

Cas simples : bases du modèle

- Spécification des séquences d'activation des états
- Comment traiter les occurrences d'événements ?
Quand ? Dans quel ordre ?
- Quelle(s) transition(s) déclencher ?
0 ou 1 dans le cas simple ;
éventuellement plusieurs dans le cas général
- Quels sont les états actifs ?
1 seul dans le cas simple
(plusieurs dans le cas des états composites ou hiérarchiques)

Occurrences d'événements

Sémantique des machines à états – cas simples

- Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- Modèle **discret** : on ne traite qu'un événement à la fois
- Les événements produits sont placés dans un *pool d'événements*
 - le pool peut être vide ou contenir plusieurs événements
 - conceptuellement, l'événement traité est *choisi arbitrairement*
une **sémantique opérationnelle** doit spécifier cet ordre
aléatoire ou *file* (=PAPS)
- *run-to-completion* : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Occurrences d'événements

Sémantique des machines à états – cas simples

- Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- Modèle **discret** : on ne traite qu'un événement à la fois
- Les événements produits sont placés dans un *pool d'événements*
 - le pool peut être vide ou contenir plusieurs événements
 - conceptuellement, l'événement traité est *choisi arbitrairement*
une **sémantique opérationnelle** doit spécifier cet ordre
aléatoire ou *file* (=PAPS)
- *run-to-completion* : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Occurrences d'événements

Sémantique des machines à états – cas simples

- Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- Modèle **discret** : on ne traite qu'un événement à la fois
- Les événements produits sont placés dans un *pool d'événements*
 - le pool peut être vide ou contenir plusieurs événements
 - conceptuellement, l'événement traité est *choisi arbitrairement*
une **sémantique opérationnelle** doit spécifier cet ordre
aléatoire ou *file* (=PAPS)
- *run-to-completion* : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Occurrences d'événements

Sémantique des machines à états – cas simples

- Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- Modèle **discret** : on ne traite qu'un événement à la fois
- Les événements produits sont placés dans un *pool d'événements*
 - le pool peut être vide ou contenir plusieurs événements
 - conceptuellement, l'événement traité est *choisi arbitrairement* une **sémantique opérationnelle** doit spécifier cet ordre aléatoire ou *file* (=PAPS)
- *run-to-completion* : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Occurrences d'événements

Sémantique des machines à états – cas simples

- Modèle **asynchrone** : deux événements ne peuvent se produire simultanément et ils sont indépendants
- Modèle **discret** : on ne traite qu'un événement à la fois
- Les événements produits sont placés dans un *pool d'événements*
 - le pool peut être vide ou contenir plusieurs événements
 - conceptuellement, l'événement traité est *choisi arbitrairement* une **sémantique opérationnelle** doit spécifier cet ordre aléatoire ou *file* (=PAPS)
- *run-to-completion* : on répercute toutes les conséquences de l'occurrence d'un événement avant d'en traiter un autre

Transitions déclenchées / traitement d'un événement

Sémantique des machines à états – cas simples

- 1 Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
- 2 Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
- 3 Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent);
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
- 4 Les effets associés aux transitions éligibles sont exécutés
- 5 L'état cible est activé

Transitions déclenchées / traitement d'un événement

Sémantique des machines à états – cas simples

- 1 Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
- 2 Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
- 3 Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent);
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
- 4 Les effets associés aux transitions éligibles sont exécutés
- 5 L'état cible est activé

Transitions déclenchées / traitement d'un événement

Sémantique des machines à états – cas simples

- ❶ Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
- ❷ Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
- ❸ Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent) ;
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
- ❹ Les effets associés aux transitions éligibles sont exécutés
- ❺ L'état cible est activé

Transitions déclenchées / traitement d'un événement

Sémantique des machines à états – cas simples

- ❶ Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
- ❷ Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
- ❸ Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent) ;
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
- ❹ Les effets associés aux transitions éligibles sont exécutés
- ❺ L'état cible est activé

Transitions déclenchées / traitement d'un événement

Sémantique des machines à états – cas simples

- ❶ Toutes les transitions étiquetées par cet événement (ou par un événement plus général) et dont l'état source est actif sont **éligibles** (= potentiellement déclenchables)
- ❷ Qu'il y ait ou non une transition éligible, l'événement est consommé (retiré du *pool*)
- ❸ Les gardes associées aux transitions éligibles sont évaluées (l'ordre est indifférent) ;
seules les transitions éligibles dont les conditions de garde sont satisfaites sont déclenchées
- ❹ Les effets associés aux transitions éligibles sont exécutés
- ❺ L'état cible est activé

Activités « internes »

- Objectif : meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classier
- `do / activity` : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- `entry / activity` : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités `do` et `exit`
- `exit / activity` : activité exécutée lorsque l'état devient inactif
- `include / activity` : permet d'invoquer un sous-diagramme d'états-transitions

Activités « internes »

- Objectif : meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classier
- `do / activity` : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- `entry / activity` : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités `do` et `exit`
- `exit / activity` : activité exécutée lorsque l'état devient inactif
- `include / activity` : permet d'invoquer un sous-diagramme d'états-transitions

Activités « internes »

- Objectif : meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classier
- `do / activity` : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- `entry / activity` : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités `do` et `exit`
- `exit / activity` : activité exécutée lorsque l'état devient inactif
- `include / activity` : permet d'invoquer un sous-diagramme d'états-transitions

Activités « internes »

- Objectif : meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classier
- `do / activity` : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- `entry / activity` : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités `do` et `exit`
- `exit / activity` : activité exécutée lorsque l'état devient inactif
- `include / activity` : permet d'invoquer un sous-diagramme d'états-transitions

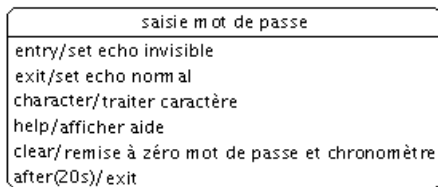
Activités « internes »

- Objectif : meilleure encapsulation de la modélisation des comportements en distinguant ce qui dépend de l'environnement (événements – transitions) et de l'état intrinsèque d'un classier
- `do / activity` : activité effectuée quand l'état est actif
elle se termine d'elle-même ou quand l'état n'est plus actif
- `entry / activity` : activité exécutée lorsque l'état devient actif
elle est exécutée après les activités des transitions d'entrée et avant les activités `do` et `exit`
- `exit / activity` : activité exécutée lorsque l'état devient inactif
- `include / activity` : permet d'invoquer un sous-diagramme d'états-transitions

Transitions « internes »

- transition attachée à un état
(doit être actif pour qu'elle soit déclenchée)
- ne provoque pas de changement d'état
donc pas de réalisation des activités internes `entry` ni `exit`
- Notation : expression d'une transition mais pas d'arc :
`nomEVT(liste param)[garde]/activites`

Activités / Transitions « internes » – Exemple



Point de choix et point de jonction

- Objectif : notation mettant en évidence les alternatives dans les modèles États–Transitions
- Pseudo-état correspondant à une **alternative** liée à l'évaluation des **conditions de garde**
- Dans un modèle bien formé, exactement une condition de garde des transitions de sortie d'un point de choix doit être valide
- Les transitions ayant comme état cible ou état source un point de choix (de jonction) sont appelées **segments de transition** (cf. sémantique machine à état)
- **Différence** : moment auquel les gardes des segments de transition en aval du point de choix sont évaluées

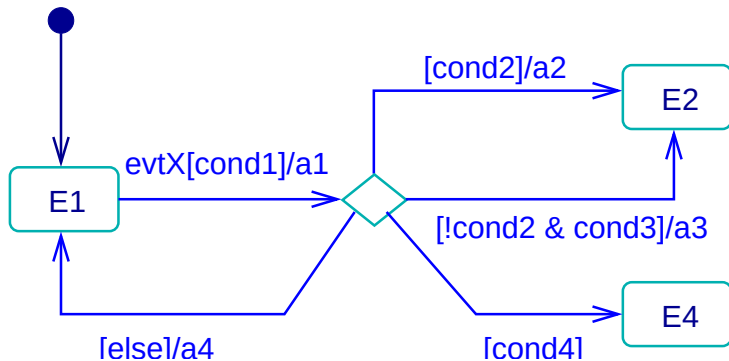
Point de choix (= de décision)

- Un pseudo-état « point de choix »
 - possède **exactement un segment de transition entrant** et au moins deux segments sortants
 - les **gardes** situées sur les segments APRÈS le point de décision sont évaluées **au moment où ce dernier est atteint**
- Il est possible d'utiliser une garde particulière, notée **[else]**, sur un des segments en sortie d'un point de choix.
 - Ce segment n'est franchissable que si les gardes des autres segments sont toutes fausses.
 - L'utilisation d'une clause **[else]** est recommandée après un point de décision car elle garantit un modèle bien formé.

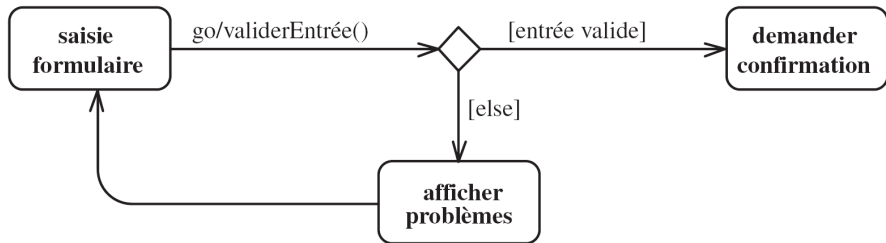
Point de choix (= de décision)

- Un pseudo-état « point de choix »
 - possède **exactement un segment de transition entrant** et au moins deux segments sortants
 - les **gardes** situées sur les segments APRÈS le point de décision sont évaluées **au moment où ce dernier est atteint**
- Il est possible d'utiliser une garde particulière, notée **[else]**, sur un des segments en sortie d'un point de choix.
 - Ce segment n'est franchissable que si les gardes des autres segments sont toutes fausses.
 - L'utilisation d'une clause **[else]** est recommandée après un point de décision car elle garantit un modèle bien formé.

Point de choix — notation (exemple)



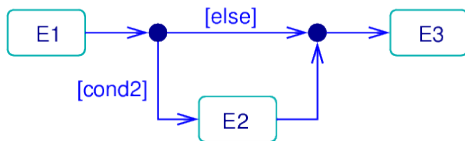
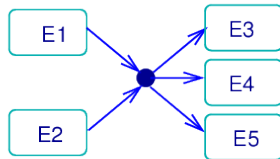
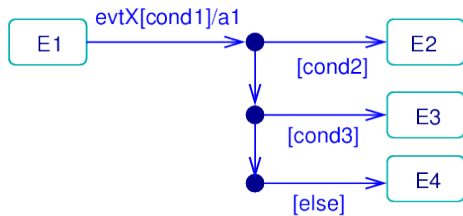
Point de choix — notation (exemple)



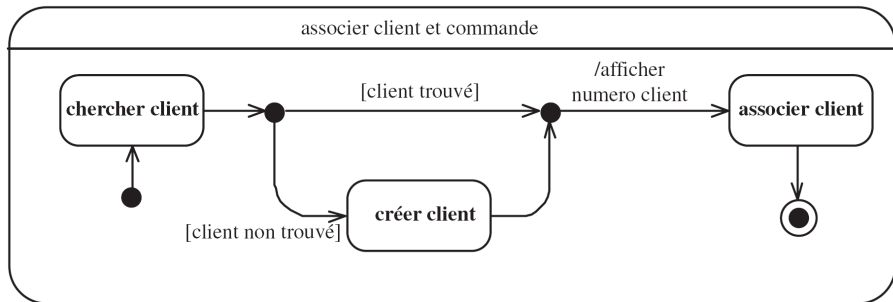
Point de jonction – définition

- Un pseudo-état « Un point de jonction »
 - peut avoir plusieurs segments de transition entrants et plusieurs segments de transition sortants
 - seuls les segments entrants peuvent être synchronisés sur des événements
 - il ne peut pas avoir d'activité interne, ni des transitions sortantes dotées de déclencheurs d'événements
 - évaluation des gardes : au moment du déclenchement du segment entrant

Point de jonction — notation (exemples)



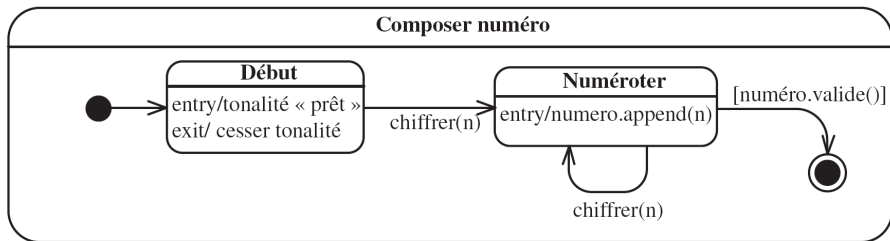
Point de jonction — notation (exemple)



État composite – Présentation

- Un état simple ne possède pas de sous-structure mais uniquement, le cas échéant, un jeu d'activités et/ou de transitions internes
- Un **état composite** est un état décomposé en *régions* contenant chacune un ou plusieurs sous-états

État composite – Exemple



Composer numéro est un état composite

État composite – Notation abrégée

- L'utilisation d'états composites permet de développer une spécification par raffinement.
- Il n'est pas nécessaire de représenter les sous-états à chaque utilisation de l'état englobant
- Une notation abrégée permet d'indiquer qu'un état est composite et que sa définition est donnée sur un autre diagramme.



État composite – transition

[1/2]

- Une transition peut avoir pour cible la frontière d'un état composite
≡ transition ayant pour **cible l'état initial** de l'état composite cible
- Une transition peut avoir pour source la frontière d'un état composite
≡ la transition **s'applique à tout sous-état** de l'état composite source.

Relation est transitive :

la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

État composite – transition

[1/2]

- Une transition peut avoir pour cible la frontière d'un état composite
≡ transition ayant pour **cible l'état initial** de l'état composite cible
- Une transition peut avoir pour source la frontière d'un état composite
≡ la transition **s'applique à tout sous-état** de l'état composite source.

Relation est transitive :

la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

État composite – transition

[1/2]

- Une transition peut avoir pour cible la frontière d'un état composite
≡ transition ayant pour **cible l'état initial** de l'état composite cible
- Une transition peut avoir pour source la frontière d'un état composite
≡ la transition **s'applique à tout sous-état** de l'état composite source.

Relation est transitive :

la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

État composite – transition

[1/2]

- Une transition peut avoir pour cible la frontière d'un état composite
≡ transition ayant pour **cible l'état initial** de l'état composite cible
- Une transition peut avoir pour source la frontière d'un état composite
≡ la transition **s'applique à tout sous-état** de l'état composite source.

Relation est transitive :

la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

État composite – transition

[1/2]

- Une transition peut avoir pour cible la frontière d'un état composite
≡ transition ayant pour **cible l'état initial** de l'état composite cible
- Une transition peut avoir pour source la frontière d'un état composite
≡ la transition **s'applique à tout sous-état** de l'état composite source.

Relation est transitive :

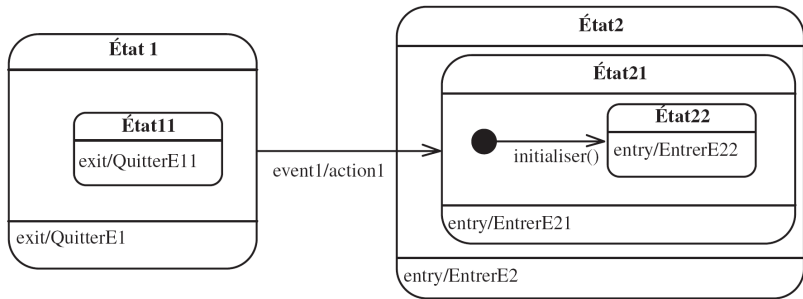
la transition est franchissable depuis tout état imbriqué, quelque soit sa profondeur.

État composite – transition

[2/2]

- Si une transition ayant pour source la frontière d'un état composite ne porte **pas de déclencheur explicite** (i.e. s'il s'agit d'une transition d'achèvement), elle est franchissable quand l'état final de l'état composite est atteint.
- Une transition peut avoir comme *source* et *cible* des états de différents niveaux d'imbrication : elle traverse donc les frontières des états composites.

État composite – transition : exemple



Depuis l'état **État 1**, la réception de l'événement *event1* produit la séquence d'activités : `QuitterE11`, `QuitterE1`, `action1`, `EntrerE2`, `EntrerE21`, `initialiser()`, `EntrerE22`, et place le système dans l'état **État22**.

Définitions

- *Parallélisme = concurrence* : évolutions indépendantes de différents éléments
- Dynamique des états est **indépendante** (= *orthogonale*)
- Modélisation : **état composite** avec régions orthogonales

Région

Ensemble d'états et de transitions

- 1 transition peut avoir un état source ou cible « traversant » l'état composite
- par déf., aucune transition entre états de régions orthogonales
- 1 et 1 seul état actif par région (si état composite englobant actif)

Définitions

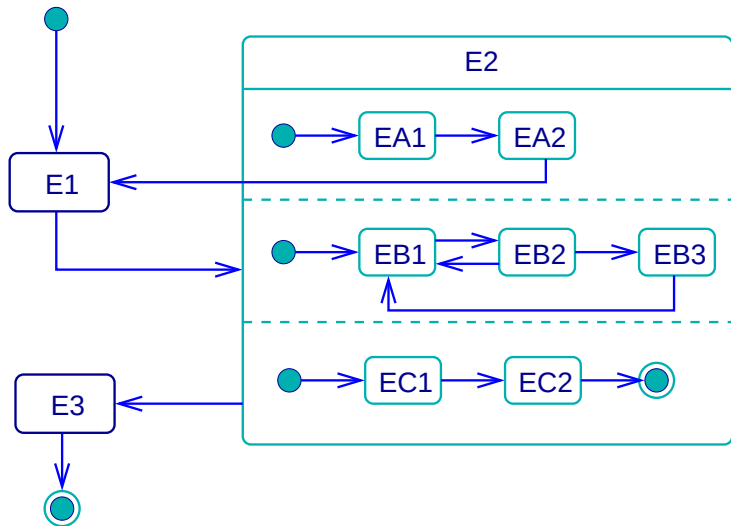
- *Parallélisme = concurrence* : évolutions indépendantes de différents éléments
- Dynamique des états est **indépendante** (= *orthogonale*)
- Modélisation : **état composite** avec régions orthogonales

Région

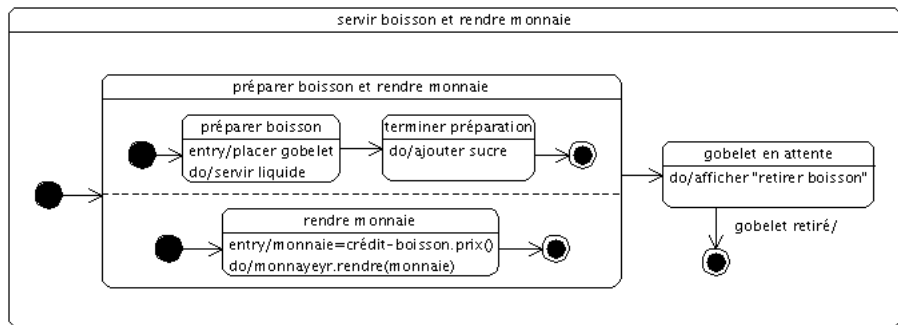
Ensemble d'états et de transitions

- 1 transition peut avoir un état source ou cible « traversant » l'état composite
- par déf., aucune transition entre états de régions orthogonales
- 1 et 1 seul état actif par région (si état composite englobant actif)

Régions orthogonales : illustration



Régions orthogonales : exemple



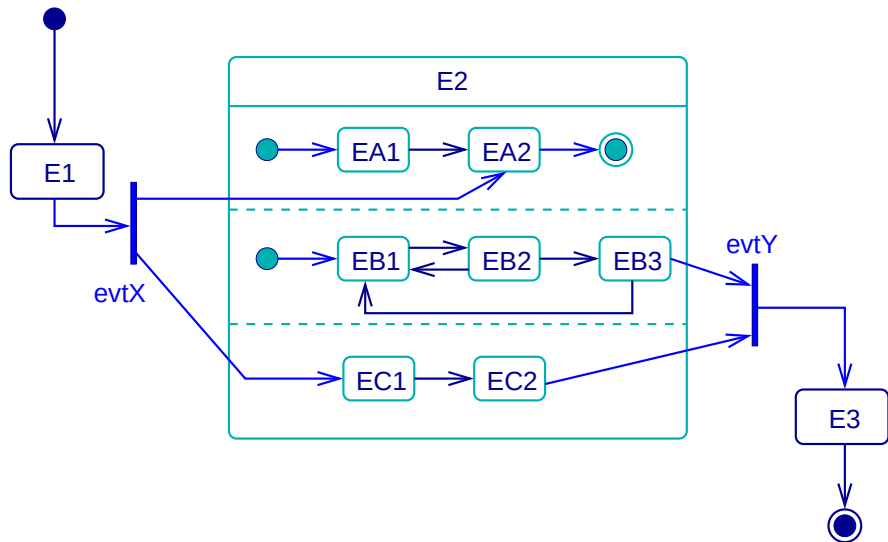
Transitions complexes

débranchement et jointure

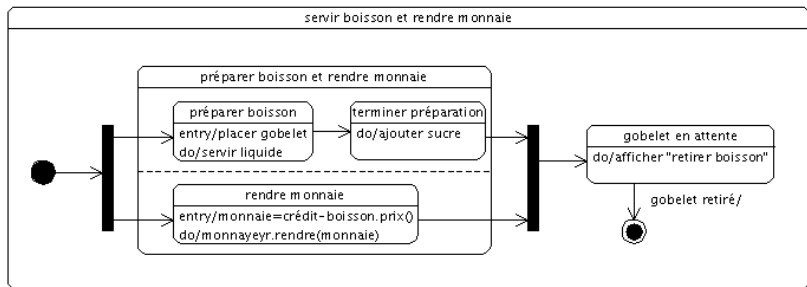
- Transition ayant plusieurs états source et/ou cible
- Une **transition entrante** active 1 et 1 seul état de chaque région
soit l'état initial (par défaut)
soit un état spécifique ciblé par la transition
et donc l'état composite (cf. *entry*)
- Une **transition sortante** désactive l'état actif de chaque région
ou l'état terminal
et donc l'état composite (cf. *exit*)
- Un **seul déclencheur** par transition complexe
(ou aucun = transition d'achèvement)

Débranchement (*fork*) – jointure (*join*)

Illustration

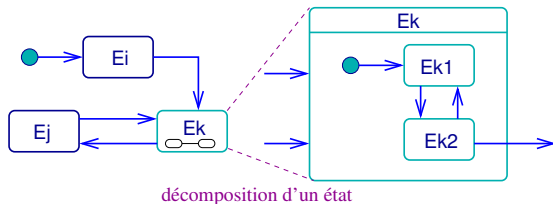


Transition complexe : exemple

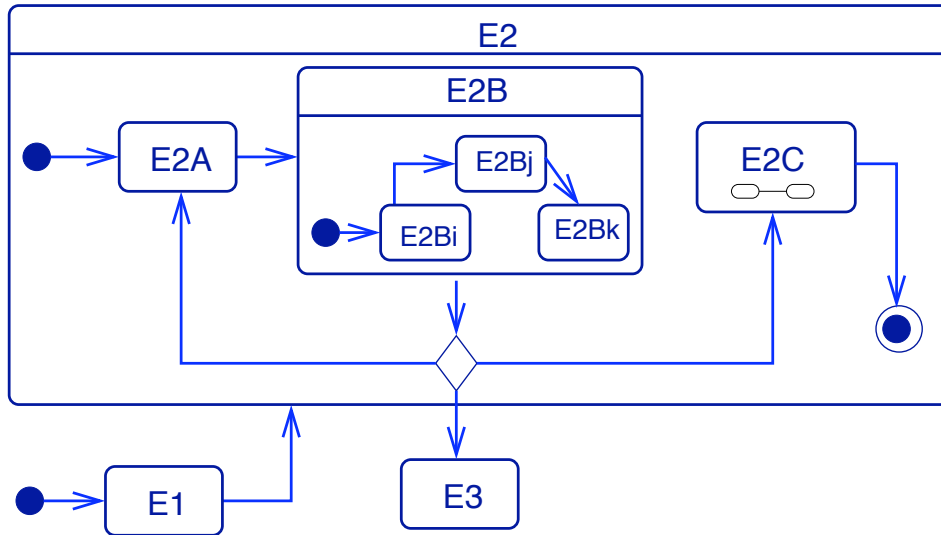


Hiérarchie : modélisation

- Décomposition d'un état en une région, elle-même éventuellement décomposée en une région
- les « sous-régions » sont donc non orthogonales :
1 seul état actif dans la hiérarchie
- Objectif : **structuration** et **raffinement** de la modélisation



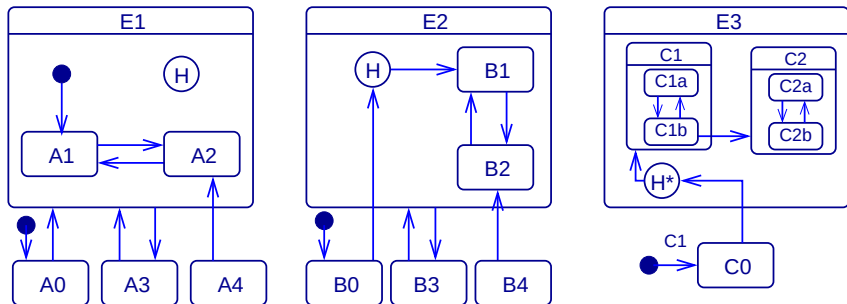
État hiérarchique : illustration



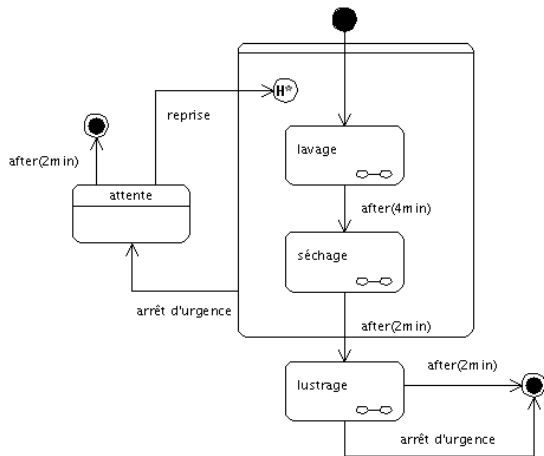
Hiérarchie : historique

- Un pseudo-état *historique* peut être associé à une région : quand elle est activée, c'est le dernier état actif qui est à nouveau activé, et non l'état initial.
- Historique *plat* H ou *profond* H* :
réactivation de l'état au niveau courant (*plat*)
et des éventuels sous-états (*profond*)

Hiérarchie – historique : illustrations



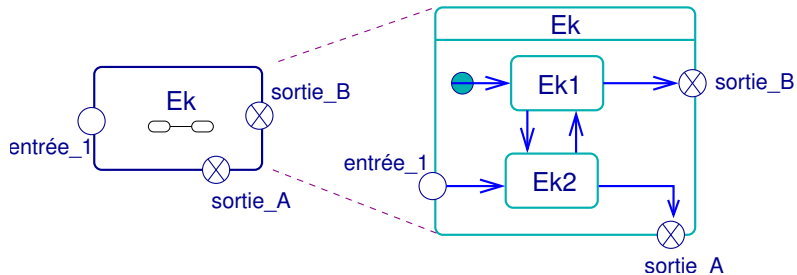
Hiérarchie – historique : exemple



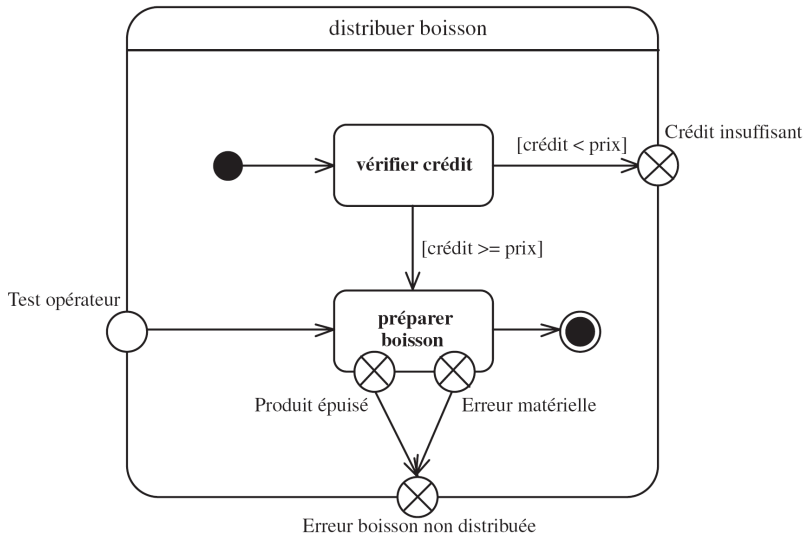
Hiérarchie : points de connexion

- Pseudo-états visibles à l'extérieur d'un état composite et référant un état « interne »
- Deux types : *point d'entrée* et *point de sortie*
- Représentations graphiques :
 - point d'entrée : cercle vide ○
 - point de sortie : cercle avec une croix ⊗

Hiérarchie – points de connexion : illustration



Hiérarchie – points de connexion : exemple



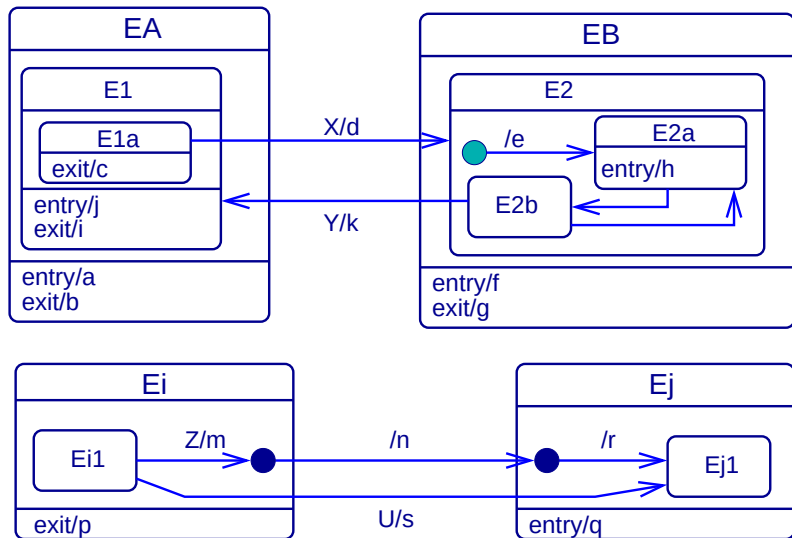
Machine à état : concepts avancés

- Déclenchement : occurrence d'un événement
étape *Run-To-Completion* (RTC)
- Évaluation des conditions de garde et exécution des effets sur plusieurs segments de transition
- Différence entre point de jonction et point de choix (étape intermédiaire : impact des effets sur les conditions de garde des segments sortants)
- *Ordre d'exécution des effets :*
 - en remontant depuis l'état source jusqu'au premier ancêtre commun,
 - puis en redescendant vers l'état cible

Machine à état : concepts avancés

- Déclenchement : occurrence d'un événement
étape *Run-To-Completion* (RTC)
- Évaluation des conditions de garde et exécution des effets sur plusieurs segments de transition
- Différence entre point de jonction et point de choix (étape intermédiaire : impact des effets sur les conditions de garde des segments sortants)
- **Ordre d'exécution des effets :**
 - en remontant depuis l'état source jusqu'au premier ancêtre commun,
 - puis en redescendant vers l'état cible

Machine à état : séquencement



Un modèle d'états-transitions pour quelles classes ?

- Toutes les classes ne requièrent pas la modélisation de leur comportement
- Identifier celles qui ont un comportement complexe :
 - Les objets de la classe réagissent différemment à l'occurrence du même événement
 - Chaque type de réaction caractérise un état particulier
 - La classe doit réaliser certaines opérations dans un ordre précis
 - Des états séquentiels permettent de préciser la chronologie forcée des opérations

Un modèle d'états-transitions pour quelles classes ?

- Toutes les classes ne requièrent pas la modélisation de leur comportement
- Identifier celles qui ont un comportement complexe :
 - Les objets de la classe réagissent différemment à l'occurrence du même événement
 - Chaque type de réaction caractérise un état particulier
 - La classe doit réaliser certaines opérations dans un ordre précis
 - Des états séquentiels permettent de préciser la chronologie forcée des opérations

Comment construire un diagramme d'états-transitions ?

- 1 Représentez la séquence d'états qui décrit le comportement nominal d'un objet, avec les transitions associées
- 2 Ajoutez progressivement les transitions qui correspondent aux comportements alternatifs ou d'erreur
- 3 Complétez les activités sur les transitions et dans les états
- 4 Structurez le diagramme en sous-états s'il devient trop complexe

Comment identifier les états d'une classe ?

Trois démarches complémentaires :

- **Expertise métier** :
 - certains états fondamentaux font partie du vocabulaire des experts
- Étude des **attributs** et des **associations** de la classe :
 - identifiez des invariants d'état
 - cherchez les valeurs seuils d'attributs qui modifient la dynamique
 - cherchez des comportements induits par l'existence ou l'absence de certains liens
- Classe par classe
 - cherchez le **diagramme d'interaction** le plus représentatif du comportement des instances
 - associez un état à chaque intervalle entre événements émis ou reçus par instance et placez les transitions

Quand utiliser les diagrammes d'états-transitions ?

Complémentarité entre modèles

- Convient bien pour décrire le **comportement d'un objet** à travers plusieurs cas d'utilisation
 - Convient mal pour décrire un comportement qui implique plusieurs objets qui collaborent
- Les diag. d'interaction décrivent bien le comportement de plusieurs objets dans un cas d'utilisation
- Les diag. d'activités montrent bien l'organisation séquentielle globale des activités de plusieurs objets et cas d'utilisation

- 1 Diagramme d'états-transitions
- 2 Diagramme d'activités
- 3 Génération de code

Objectifs du modèle

Le modèle des activités – *activity chart*

- Objectifs – utilisations en modélisation :
 - **Processus métier** : aspects conceptuels et organisationnels
Réalisation des cas d'utilisation
 - **Conception du logiciel** : réalisation des actions de base et structures de contrôle
Algorithmique des méthodes
- Modélisation du comportement du système, en mettant l'accent sur les traitements effectués : les **activités du système**
- Approche **flux de contrôle** et **flux de données**

Objectifs du modèle

Le modèle des activités – *activity chart*

- Objectifs – utilisations en modélisation :
 - **Processus métier** : aspects conceptuels et organisationnels
Réalisation des cas d'utilisation
 - **Conception du logiciel** : réalisation des actions de base et structures de contrôle
Algorithmique des méthodes
- Modélisation du comportement du système, en mettant l'accent sur les traitements effectués : les **activités du système**
- Approche **flux de contrôle** et **flux de données**

Objectifs du modèle

Le modèle des activités – *activity chart*

- Objectifs – utilisations en modélisation :
 - **Processus métier** : aspects conceptuels et organisationnels
Réalisation des cas d'utilisation
 - **Conception du logiciel** : réalisation des actions de base et structures de contrôle
Algorithmique des méthodes
- Modélisation du comportement du système, en mettant l'accent sur les traitements effectués : les **activités du système**
- Approche **flux de contrôle** et **flux de données**

Éléments de modélisation

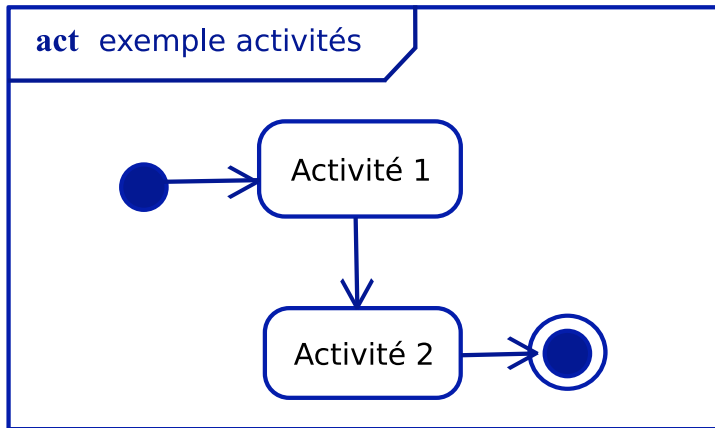
- Éléments de base : activité et action
- Flot de contrôle et flot d'objet
- Nœud de contrôle
- Autres éléments de contrôle

Activité : définition

Élément de comportement d'un classier

- Traitement exécutable par une entité du système
- L'exécution d'une activité conduit à l'exécution d'actions en respectant un certain séquençement
- Dans ce modèle, l'exécution d'une activité n'est pas instantanée
- et il n'est pas atomique
- Des activités peuvent s'exécuter en parallèle

Activités : notation



Action : définition

- **Unité fondamentale d'exécution d'une activité** : plus petit élément de modélisation d'un comportement en UML
- Exécutée dans le *contexte* d'un classier
- L'exécution d'une action correspond à une **transformation d'éléments du système**
 - **Syst. informatique** : lecture / modification de variables ou de propriétés du classier, appel d'une opération, création de nouveaux objets
 - **Processus métier** : traitement d'une information, valeur ajoutée au processus
- Une action est caractérisée par un *effet* : **description textuelle** des conséquences de son exécution sur les éléments du système

Action : définition

- **Unité fondamentale d'exécution d'une activité** : plus petit élément de modélisation d'un comportement en UML
- Exécutée dans le *contexte* d'un classifier
- L'exécution d'une action correspond à une **transformation d'éléments du système**
 - *Syst. informatique* : lecture / modification de variables ou de propriétés du classifier, appel d'une opération, création de nouveaux objets
 - *Processus métier* : traitement d'une information, valeur ajoutée au processus
- Une action est caractérisée par un *effet* : **description textuelle** des conséquences de son exécution sur les éléments du système

Action : définition

- **Unité fondamentale d'exécution d'une activité** : plus petit élément de modélisation d'un comportement en UML
- Exécutée dans le *contexte* d'un classier
- L'exécution d'une action correspond à une **transformation d'éléments du système**
 - **Syst. informatique** : lecture / modification de variables ou de propriétés du classier, appel d'une opération, création de nouveaux objets
 - **Processus métier** : traitement d'une information, valeur ajoutée au processus
- Une action est caractérisée par un *effet* : **description textuelle** des conséquences de son exécution sur les éléments du système

Action : définition

- **Unité fondamentale d'exécution d'une activité** : plus petit élément de modélisation d'un comportement en UML
- Exécutée dans le *contexte* d'un classifier
- L'exécution d'une action correspond à une **transformation d'éléments du système**
 - **Syst. informatique** : lecture / modification de variables ou de propriétés du classifier, appel d'une opération, création de nouveaux objets
 - **Processus métier** : traitement d'une information, valeur ajoutée au processus
- Une action est caractérisée par un *effet* : **description textuelle** des conséquences de son exécution sur les éléments du système

Action : propriétés

- **Contraintes** : conditions devant être satisfaites
(ne pas confondre avec une condition de garde)
 - `localPrecondition` : quand l'action démarre
 - `localPostcondition` : quand l'action est terminée

Leur traitement est un point de variation sémantique

- **Entrées et Sorties** : *pins*
ensembles ordonnés de variables nommées
 - *input pin*
 - *output pin*
- **Exception(s)** : *exception pin*
Si une exception survient, l'action est abandonnée
et aucune sortie n'est générée

Action : propriétés

- **Contraintes** : conditions devant être satisfaites
(ne pas confondre avec une condition de garde)
 - `localPrecondition` : quand l'action démarre
 - `localPostcondition` : quand l'action est terminée

Leur traitement est un point de variation sémantique

- **Entrées et Sorties** : *pins*
ensembles ordonnés de variables nommées
 - *input pin*
 - *output pin*

- **Exception(s)** : *exception pin*
Si une exception survient, l'action est abandonnée
et aucune sortie n'est générée

Action : propriétés

- **Contraintes** : conditions devant être satisfaites
(ne pas confondre avec une condition de garde)
 - `localPrecondition` : quand l'action démarre
 - `localPostcondition` : quand l'action est terminée

Leur traitement est un point de variation sémantique

- **Entrées et Sorties** : *pins*
ensembles ordonnés de variables nommées
 - *input pin*
 - *output pin*
- **Exception(s)** : *exception pin*
Si une exception survient, l'action est abandonnée
et aucune sortie n'est générée

Action de communication : notations stéréotypés

De l'événementiel dans le procédural

- *accept event* :
son exécution bloque le flot d'exécution
tant qu'un événement n'est pas détecté,
généralement un type de signal
- *accept time event* :
attente qu'une expression temporelle se vérifie
Ex. : "10 s", "tous les jours"
- *send signal* :
création d'un message et transmission de ce signal
au destinataire

Action de communication : notations stéréotypés

De l'événementiel dans le procédural

- *accept event* :
son exécution bloque le flot d'exécution
tant qu'un événement n'est pas détecté,
généralement un type de signal
- *accept time event* :
attente qu'une expression temporelle se vérifie
Ex. : "10 s", "tous les jours"
- *send signal* :
création d'un message et transmission de ce signal
au destinataire

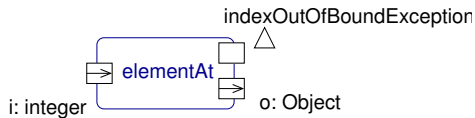
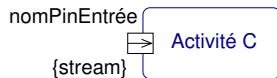
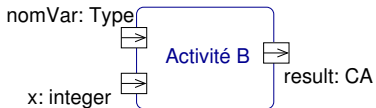
Action de communication : notations stéréotypés

De l'événementiel dans le procédural

- *accept event* :
son exécution bloque le flot d'exécution
tant qu'un événement n'est pas détecté,
généralement un type de signal
- *accept time event* :
attente qu'une expression temporelle se vérifie
Ex. : "10 s", "tous les jours"
- *send signal* :
création d'un message et transmission de ce signal
au destinataire

Pins : Notations

Entrée, sortie, exception



Flot de contrôle et flot de données

Définition

- Flot de contrôle : description des séquences d'exécution des activités
- Flot de données : passage d'une information produite par une activité à une activité qui la consomme

Exécution d'une action

- l'exécution d'une action « consomme » les flots de contrôle et d'objets entrants
- elle ne peut donc commencer que s'ils sont valides
- quand elle est terminée, elle valorise ses flots de données de sortie (sauf si exception) et « passe » le contrôle aux activités cibles

Flot de contrôle et flot de données

Définition

- Flot de contrôle : description des séquences d'exécution des activités
- Flot de données : passage d'une information produite par une activité à une activité qui la consomme

Exécution d'une action

- l'exécution d'une action « consomme » les flots de contrôle et d'objets entrants
- elle ne peut donc commencer que s'ils sont valides
- quand elle est terminée, elle valorise ses flots de données de sortie (sauf si exception) et « passe » le contrôle aux activités cibles

Objet d'un flot de données (*ObjectNode*)

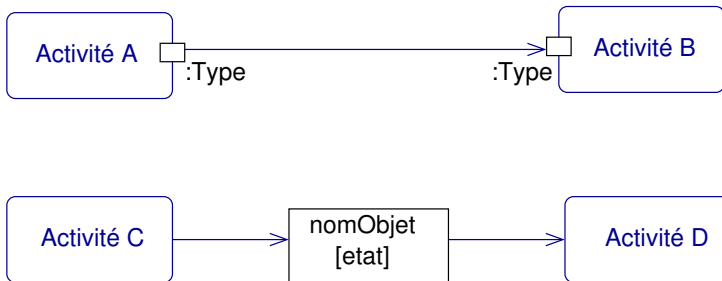
Élément d'information véhiculé par un **flot de données**
(*Object flow*)

Propriétés

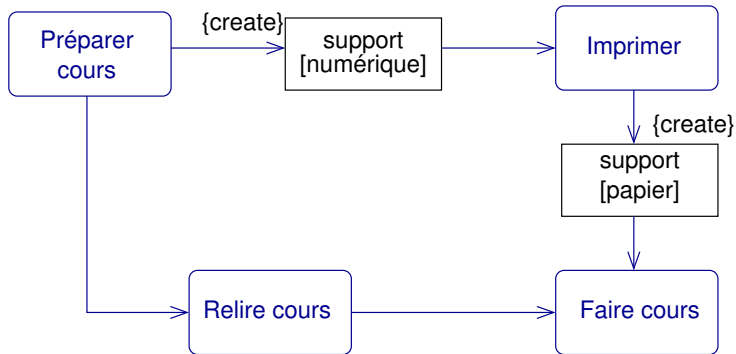
- instance d'un classifier (obligatoire)
- état(s) du classifier requis à ce point de l'exécution
- ordre de sélection ($\{\text{ordering} = \text{FIFO}\}$ par défaut)
- nombre maximum d'objets sur le flot ($\{\text{upperBound} = n\}$)
(s'il est atteint, l'activité est bloquée)

Objet d'un flot de données : notation

Deux notations possibles



Combinaison d'un flot de contrôle et d'un flot d'objet



Nœuds de contrôle

- Nœud initial, final et de terminaison
- Nœud de décision et nœud de fusion (structure conditionnelle)
- Nœud de débranchement et de jointure (*fork – join*)

Nœud initial, final et de terminaison

Nœud initial

indique la première activité à réaliser :

initialisation du flot de contrôle

notation : idem état-transition

Nœud final

- **Activité** (*activity final node*) :

indique que le flot de contrôle est terminé :

TOUS les flots sont alors désactivés

notation : idem état-transition

- **Flot** (*flow final node*) :

UN des flots de contrôle s'achève (cas parallélisme)

notation : point de sortie états-transitions ⊗

Nœud initial, final et de terminaison

Nœud initial

indique la première activité à réaliser :

initialisation du flot de contrôle

notation : idem état-transition

Nœud final

- **Activité** (*activity final node*) :

indique que le flot de contrôle est terminé :

TOUS les flots sont alors désactivés

notation : idem état-transition

- **Flot** (*flow final node*) :

UN des flots de contrôle s'achève (cas parallélisme)

notation : point de sortie états-transitions ⊗

Nœud de décision et nœud de fusion

nœud de contrôled'une alternative

● Nœud de décision

- Choix exclusif entre plusieurs membres d'une alternative
- 1 arc entrant et plusieurs arcs sortants (1 par choix)
- 1 *DecisionInput* $\equiv$ « règle de décision »
- Chaque arc sortant correspond à une valeur possible de *decision input* (ou [else])
expression du même type que *DecisionInput*
- Dans un modèle bien formé, l'alternative couvre tous les cas

● Nœud de fusion : point où des chemins alternatifs se rejoignent

● Nœud de décision–fusion : syntaxiquement correct

Nœud de décision et nœud de fusion

nœud de contrôled'une alternative

- **Nœud de décision**

- Choix exclusif entre plusieurs membres d'une alternative
- 1 arc entrant et plusieurs arcs sortants (1 par choix)
- 1 *DecisionInput* $\equiv$ « règle de décision »
- Chaque arc sortant correspond à une valeur possible de *decision input* (ou [else])
expression du même type que *DecisionInput*
- Dans un modèle bien formé, l'alternative couvre tous les cas

- **Nœud de fusion** : point où des chemins alternatifs se rejoignent

- **Nœud de décision–fusion** : syntaxiquement correct

Nœud de décision et nœud de fusion

nœud de contrôled'une alternative

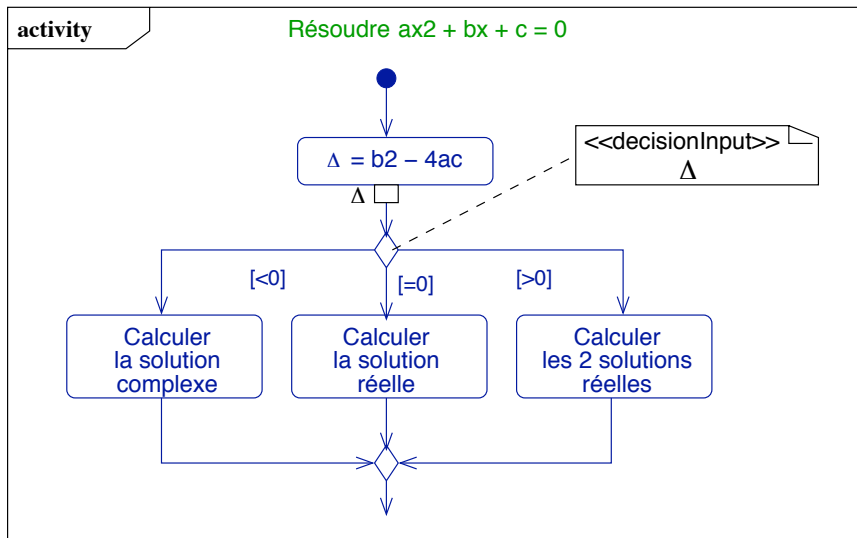
- **Nœud de décision**

- Choix exclusif entre plusieurs membres d'une alternative
- 1 arc entrant et plusieurs arcs sortants (1 par choix)
- 1 *DecisionInput* $\equiv$ « règle de décision »
- Chaque arc sortant correspond à une valeur possible de *decision input* (ou [else])
expression du même type que *DecisionInput*
- Dans un modèle bien formé, l'alternative couvre tous les cas

- **Nœud de fusion** : point où des chemins alternatifs se rejoignent

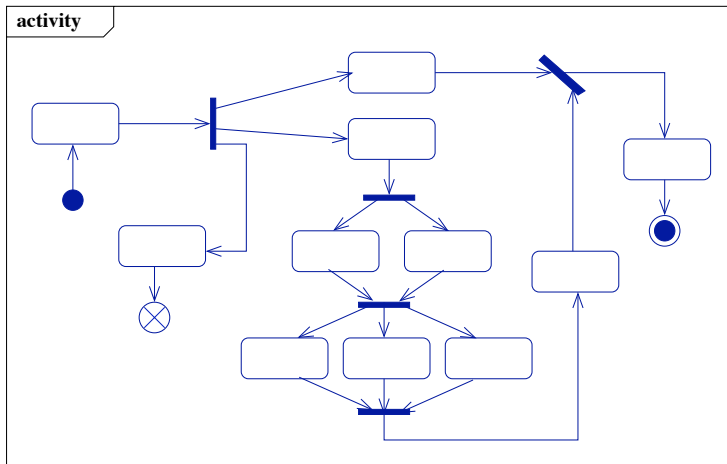
- **Nœud de décision–fusion** : syntaxiquement correct

Nœud de décision et nœud de fusion : exemple



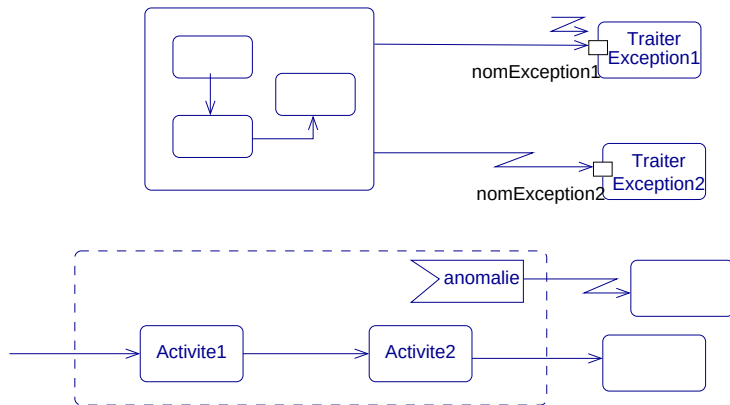
Nœud de débranchement et de jointure

(*fork – join*)



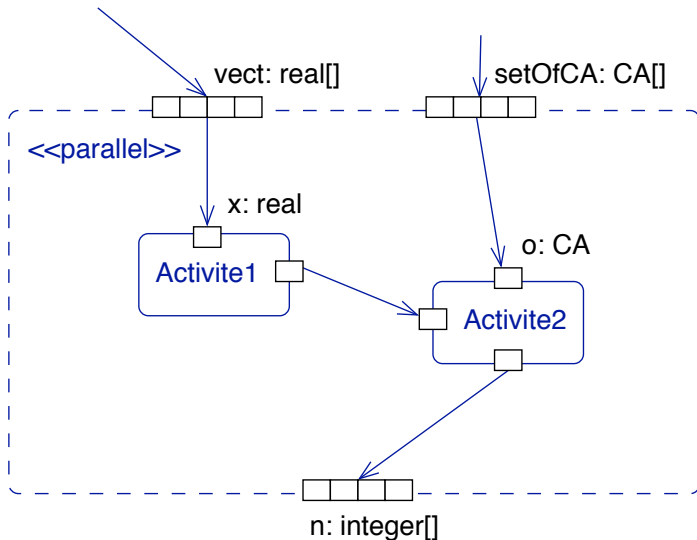
Interruption d'une activité (exception)

Différents styles de notation



Région d'expansion

Exécution d'une activité sur les éléments d'une collection



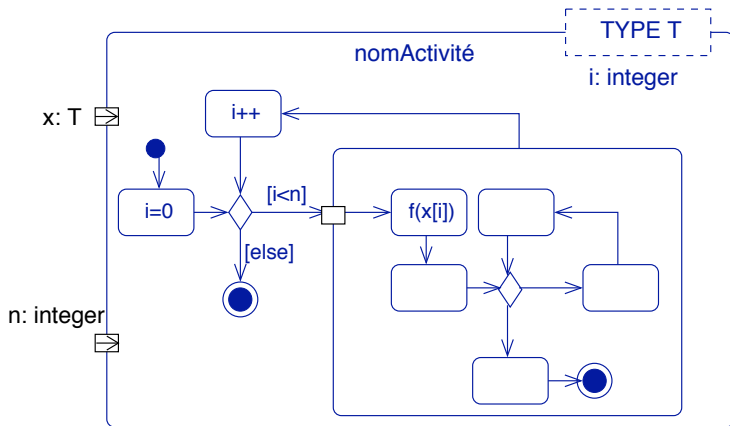
Région d'expansion : sémantique de l'exécution

- **Nœud d'expansion** : collection d'éléments traités par les activités de la région.
 - Nœud d'expansion d'entrée : la collection est produite par une activité en amont ; les activités de la région d'expansion traitent chaque élément de la collection
 - Nœud d'expansion de sortie : chaque exécution de la région produit un élément de la collection
- **Modes d'exécution** :
 - «parallel» : de manière indépendante
 - «iterative» : nécessairement en séquence si collection ordonnée, alors dans l'ordre
 - «streaming» : une exécution unique pour tous les éléments de la collection

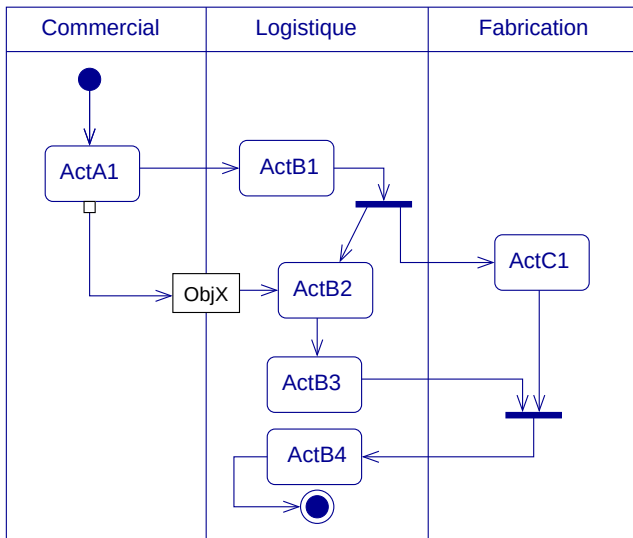
Région d'expansion : sémantique de l'exécution

- **Nœud d'expansion** : collection d'éléments traités par les activités de la région.
 - Nœud d'expansion d'entrée : la collection est produite par une activité en amont ; les activités de la région d'expansion traitent chaque élément de la collection
 - Nœud d'expansion de sortie : chaque exécution de la région produit un élément de la collection
- **Modes d'exécution** :
 - «parallel» : de manière indépendante
 - «iterative» : nécessairement en séquence si collection ordonnée, alors dans l'ordre
 - «streaming» : une exécution unique pour tous les éléments de la collection

Hiérarchie d'activités : illustration



Partition d'activités : exemple



- 1 Diagramme d'états-transitions
- 2 Diagramme d'activités
- 3 Génération de code

Modèle UML : traduction dans un langage de programmation

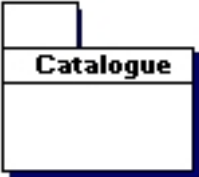
Problématique :

- UML est indépendant des langages de programmation et des technologies
- Comment implémenter (=coder) les modèles ?
- Certains éléments de modélisation sont communs à UML et aux LPO
 - Ont-ils la même sémantique ?
 - Sinon, comment assurer la traduction de l'un dans l'autre ?
 - Existe-t'il une et une seule traduction possible ?
 - La traduction est-elle « réversible » ?
- **Définition de *profiles* adaptés**

Package UML

- Soit un Package de nom "pkgName"
- Dans le répertoire du contexte courant, création d'un répertoire pkgName (**Java** et **C++**)
- **Java** (dans chaque fichier correspondant aux classes du package) :
package pkgName;
- **C++** :
 - Un répertoire pkgName dans l'arborescence include
 - Un répertoire pkgName dans l'arborescence src
 - Une bibliothèque dans lib (p. ex. libpkgName.so)
 - namespace pkgName {...};

Package UML

UML	Java
 A UML Package Diagram showing a package named 'Catalogue'. The package is represented by a rectangle with a tab on the top-left corner. The name 'Catalogue' is written in bold text on the tab.	<code>package Catalogue</code> <code>...</code>
	C++ <code>namespace Catalogue</code> <code>{</code> <code>...</code> <code>}</code>

Traduction d'une classe UML

Soit une Class CName du package pkgName

- Un fichier par classe
(sauf pour les classes contenues dans une autre classe)
- **Java** : fichier CName.java dans le répertoire du package
package pkgName;
public class CName {...}
- **C++** :
 - CName.h dans l'arborescence include (rép. du pkgName)
 - CName.cpp dans l'arborescence src (rép. du pkgName)
 - fichier CName.o (bibliothèque libPkgName.so)
 - namespace pkgName {
 class CName {...};
}

Traduction d'une classe UML

Soit une Class CName du package pkgName

- Un fichier par classe
(sauf pour les classes contenues dans une autre classe)
- **Java** : fichier CName.java dans le répertoire du package
package pkgName;
public class CName {...}
- **C++** :
 - CName.h dans l'arborescence include (rép. du pkgName)
 - CName.cpp dans l'arborescence src (rép. du pkgName)
 - fichier CName.o (bibliothèque libPkgName.so)
 - namespace pkgName {
 class CName {...};
}

Traduction d'une classe UML

Soit une Class CName du package pkgName

- Un fichier par classe
(sauf pour les classes contenues dans une autre classe)
- **Java** : fichier CName.java dans le répertoire du package
package pkgName;
public class CName {...}
- **C++** :
 - CName.h dans l'arborescence include (rép. du pkgName)
 - CName.cpp dans l'arborescence src (rép. du pkgName)
 - fichier CName.o (bibliothèque libPkgName.so)
 - namespace pkgName {
 class CName {...};
}

Cas particuliers

- Classe abstraite

- Java :

- public abstract class AA { ... }

- C++ :

- au moins une opération abstraite

- Interface :

- Java :

- public interface IJ { ... }

- C++ :

- classe dont toutes les opérations sont abstraites

Cas particuliers

- Classe abstraite

- Java :

- public abstract class AA { ... }

- C++ :

- au moins une opération abstraite

- Interface :

- Java :

- public interface IJ { ... }

- C++ :

- classe dont toutes les opérations sont abstraites

Opération d'une classe UML

- **Operation** *vs* **Method**

- Operation : spécification du service (déclaration)
- Method : définition de l'opération
- Une Operation sans Method associée est *abstraite*

- **Paramètres** :

- Type : voir traduction des types de base
- Direction : in, out, inout, return

Traduction des types de base UML en Java : integer, et real

- `« type » integer – TaggedValue {size}, {wrapped}`

	—	{wrapped}
—	int	Integer
{size,long}	long	Long
{size,short}	short	Short
{size,byte}	byte	Byte

- `« type » real – TaggedValue {size}, {wrapped}`

	—	{wrapped}
—	float	Float
{size,long}	double	Double

Traduction des types de base UML en Java : boolean, string et char

- « type » boolean – TaggedValue {wrapped}

—	{wrapped}
boolean	Boolean

- « type » string

String

- « type » char – TaggedValue {wrapped}

—	{wrapped}
char	Character

Traduction des types de base UML en C++ : integer, et real

- « type » integer – TaggedValue {size}, {unsigned}

	—	{unsigned}
—	int	unsigned int
{size,long}	long	unsigned long
{size,extraLong}	long long	unsigned long long
{size,short}	short	unsigned short

- « type » real – TaggedValue {size}

—	float
{size,long}	double
{size,extraLong}	long double

Traduction des types de base UML en C++ : boolean, string et char

- `<< type >> boolean :`
`bool`
- `<< type >> string :`
`string`
`#include <string>`
- `<< type >> char :`
`char`

Attributs d'une classe

Principes généraux de la transformation

- **Principe de transformation :**

- **implémentation non accessible** en dehors de la classe
- **accessibilité** (lecture, modification) assurée par des méthodes dont l'accessibilité correspond à la **visibilité du modèle de classe**

- **Type :** voir traduction des types de base UML

- **Cas particuliers :**

- attribut simple
- attribut de classe
- attribut dérivé
- attribut à multiples valeurs

- **Invariant :**

maintien de l'intégrité des valeurs attributs

Attribut UML : traduction en C++

Exemple d'un attribut attrName : AttrType protégé

```
class CName {  
    protected :  
        inline AttrType getAttrName(void) const ;  
        inline void setAttrName(AttrType value) ;  
        inline virtual void _invariant(void) ;  
    private :  
        AttrType _attrName ;  
};  
  
inline AttrType CName : :getAttrName(void) const {  
    return _attrName ;  
}  
  
inline void CName : :setAttrName(AttrType value) {  
    _attrName = value ;  
    _invariant() ;  
}  
  
inline void CName : :_invariant(void) {  
    // Begin  
    // End  
}
```

Attribut UML : traduction en C++

Exemple d'un attribut attrName : AttrType protégé

```
class CName {  
    protected :  
        inline AttrType getAttrName(void) const ;  
        inline void setAttrName(AttrType value) ;  
        inline virtual void _invariant(void) ;  
    private :  
        AttrType _attrName ;  
};  
inline AttrType CName : :getAttrName(void) const {  
    return _attrName ;  
}  
inline void CName : :setAttrName(AttrType value) {  
    _attrName = value ;  
    _invariant() ;  
}  
inline void CName : :_invariant(void) {  
    // Begin  
    // End  
}
```

Attribut UML : traduction en C++

Exemple d'un attribut attrName : AttrType protégé

```
class CName {  
    protected :  
        inline AttrType getAttrName(void) const ;  
        inline void setAttrName(AttrType value) ;  
        inline virtual void _invariant(void) ;  
    private :  
        AttrType _attrName ;  
};  
inline AttrType CName : :getAttrName(void) const {  
    return _attrName ;  
}  
inline void CName : :setAttrName(AttrType value) {  
    _attrName = value ;  
    _invariant() ;  
}  
inline void CName : :_invariant(void) {  
    // Begin  
    // End  
}
```


Java : attributs de la classe Plant

+quantity: real = 0

«invariant» {quantity $\in$ [0, maxQuantity]}

- Implémentation privée (cf. règle de codage) :
private double quantity;
- Accesseur public (cf. modèle) :
public double getQuantity() {
 return quantity;
}
- Modificateur : visibilité et maintien de l'invariant
public void setQuantity(double quantity) {
 quantity = Math.max(quantity, 0.0);
 quantity = Math.min(quantity, maxQuantity);
 this.quantity = quantity;
}

Java : attributs de la classe Plant

+quantity: real = 0

«invariant» {quantity $\in$ [0, maxQuantity]}

- **Implémentation privée** (cf. règle de codage) :
`private double quantity;`
- **Accesseur public** (cf. modèle) :
`public double getQuantity() {
 return quantity;
}`
- **Modificateur** : visibilité et maintien de l'invariant
`public void setQuantity(double quantity) {
 quantity = Math.max(quantity, 0.0);
 quantity = Math.min(quantity, maxQuantity);
 this.quantity = quantity;
}`

Java : attributs de la classe Plant

+quantity: real = 0

«invariant» {quantity $\in$ [0, maxQuantity]}

- **Implémentation privée** (cf. règle de codage) :
`private double quantity;`
- **Accesseur public** (cf. modèle) :
`public double getQuantity() {
 return quantity;
}`
- **Modificateur** : visibilité et maintien de l'invariant
`public void setQuantity(double quantity) {
 quantity = Math.max(quantity, 0.0);
 quantity = Math.min(quantity, maxQuantity);
 this.quantity = quantity;
}`

Java : attributs de la classe Plant

+quantity: real = 0

«invariant» {quantity $\in$ [0, maxQuantity]}

- **Implémentation privée** (cf. règle de codage) :
`private double quantity;`
- **Accesseur public** (cf. modèle) :
`public double getQuantity() {
 return quantity;
}`
- **Modificateur** : visibilité et maintien de l'invariant
`public void setQuantity(double quantity) {
 quantity = Math.max(quantity, 0.0);
 quantity = Math.min(quantity, maxQuantity);
 this.quantity = quantity;
}`

Java : attribut d'instance en lecture seule

Exemple de la classe Plant

```
+growthRate: real = 0.01 {readOnly}
```

```
public Plant(..., double growthRate) {  
    super(...);  
    this.growthRate = growthRate;  
}
```

```
public Plant(...) {  
    this(..., 0.01 /* initial value of growthRate */);  
}
```

```
private final double growthRate;
```

```
public double getGrowthRate() { return growthRate; }
```

Java : attribut de classe

Exemple de la classe Plant

+maxQuantity: real = 1.0 {readOnly}

```
private static double maxQuantity = 1;
```

```
public static double getMaxQuantity() {  
    return maxQuantity;  
}
```

Java : attribut dérivé

Exemple l'attribut age de la classe Animal

```
# birthdate: real = Clock.time {readOnly}  
+/age: real = Clock.time - birthdate
```

```
private double birthdate;
```

```
protected double getBirthdate() {  
    return birthday;  
}
```

```
// — Derived attribute 'age'
```

```
public double getAge() {  
    return Clock.getTime() - this.getBirthdate();  
}
```

Java : attribut dérivé

Exemple l'attribut age de la classe Animal

```
# birthdate: real = Clock.time {readOnly}  
+/age: real = Clock.time - birthdate
```

```
private double birthdate;
```

```
protected double getBirthdate() {  
    return birthday;  
}
```

```
// — Derived attribute 'age'
```

```
public double getAge() {  
    return Clock.getTime() - this.getBirthdate();  
}
```


Java : attribut dérivé

Exemple l'attribut age de la classe Animal

```
# birthdate: real = Clock.time {readOnly}  
+/age: real = Clock.time - birthdate
```

```
private double birthdate;
```

```
protected double getBirthdate() {  
    return birthday;  
}
```

```
// — Derived attribute 'age'
```

```
public double getAge() {  
    return Clock.getTime() - this.getBirthdate();  
}
```

Java : attribut à multiples valeurs

Exemple de la classe SquareCell

+position: real[2]

- Traduction de `real` sous forme d'un *wrapper* :

```
private Vector<Double> position = new Vector<Double>(2);  
public SquareCell(double x, double y) {  
    position.set(0, x);  
    position.set(1, y);  
}
```

- Traduction de `real` sous forme d'un *type de base* :

```
private double[] position = new double[2];  
public SquareCell(double x, double y) {  
    position[0] = x;  
    position[1] = y;  
}
```

Java : attribut à multiples valeurs

Exemple de la classe SquareCell

+position: real[2]

- Traduction de `real` sous forme d'un *wrapper* :

```
private Vector<Double> position = new Vector<Double>(2);  
public SquareCell(double x, double y) {  
    position.set(0, x);  
    position.set(1, y);  
}
```

- Traduction de `real` sous forme d'un **type de base** :

```
private double[] position = new double[2];  
public SquareCell(double x, double y) {  
    position[0] = x;  
    position[1] = y;  
}
```

Java : attribut à multiples valeurs

Exemple de la classe SquareCell

Masquage de l'implémentation

- *wrapper* :

```
public double getPosition(int index) {  
    return position.elementAt(index);  
}  
  
public void setPosition(int index, double value) {  
    position.set(index, value);  
}
```

- type de base :

```
public double getPosition(int index) {  
    return position[index];  
}  
  
void setPosition(int index, double value) {  
    position[index] = value;  
}
```

Java : attribut à multiples valeurs

Exemple de la classe SquareCell

Masquage de l'implémentation

- *wrapper* :

```
public double getPosition(int index) {  
    return position.elementAt(index);  
}  
  
public void setPosition(int index, double value) {  
    position.set(index, value);  
}
```

- type de base :

```
public double getPosition(int index) {  
    return position[index];  
}  
  
void setPosition(int index, double value) {  
    position[index] = value;  
}
```

Instanciation d'une classe UML : constructeurs

- **Constructeurs implicites :**

- nom de la méthode : nom de la classe
- valeurs initiales des attributs
- construction des conteneurs pour l'implémentation des attributs à valeurs multiples et des associations de multiplicité > 1
- action(s) des états initiaux (cf. modèle état-transition)
- Cohérence des constructeurs

- **Constructeurs explicites :**

Operation stéréotypée `<<create>>`

Java : exemple de la classe Clock

constructeurs

```
package simLife;
```

```
public class Clock {
```

```
    public Clock(double initialTime, double deltaT, double timeMax) {  
        this.time = initialTime;  
        this.deltaT = deltaT;  
        this.timeMax = timeMax;  
        // ...  
    }
```

```
    public Clock(double initialTime, double deltaT) {  
        this(initialTime, deltaT, Double.MAX_VALUE);  
    }
```

```
    public Clock() {  
        this(0.0, 1.0, Double.MAX_VALUE);  
    }
```

C++ : exemple de la classe CName

constructeurs

Soit CName ayant deux attributs :

attr1: integer = 0

attr2: string = ""

// File : CName.h

...

class CName {

public :

CName(void);

CName(const CName & instanceOfCName);

const CName & operator = (const CName & instanceOfCName);

virtual ~ CName(void);

...

protected :

virtual void _copy(void);

...

};

C++ : exemple de la classe CName (CName.cpp)

```
CName : :CName(void) : attr1(0), attr2(" ") {  
    // Begin  
    // End  
}  
  
CName : : CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
  
const CName & CName : :operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
  
CName : :~ CName(void) {  
    // Begin  
    // End  
}  
  
void CName : :_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```

C++ : exemple de la classe CName (CName.cpp)

```
CName : :CName(void) : attr1(0), attr2("") {  
    // Begin  
    // End  
}  
CName : : CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
const CName & CName : :operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
CName : :~ CName(void) {  
    // Begin  
    // End  
}  
void CName : :_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```

C++ : exemple de la classe CName (CName.cpp)

```
CName : :CName(void) : attr1(0), attr2("") {  
    // Begin  
    // End  
}  
CName : : CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
const CName & CName : :operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
CName : :~ CName(void) {  
    // Begin  
    // End  
}  
void CName : :_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```

C++ : exemple de la classe CName (CName.cpp)

```
CName : :CName(void) : attr1(0), attr2("") {  
    // Begin  
    // End  
}  
CName : : CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
const CName & CName : :operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
CName : :~ CName(void) {  
    // Begin  
    // End  
}  
void CName : :_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```

C++ : exemple de la classe CName (CName.cpp)

```
CName : :CName(void) : attr1(0), attr2("") {  
    // Begin  
    // End  
}  
CName : : CName(const CName & instanceOfCName) {  
    _copy(instanceOfCName);  
}  
const CName & CName : :operator = (const CName & instanceOfCName) {  
    if (this != & instanceOfCName) _copy(instanceOfCName);  
    return * this;  
}  
CName : :~ CName(void) {  
    // Begin  
    // End  
}  
void CName : :_copy(const CName & instanceOfCName) {  
    this->setAttr1(instanceOfCName.getAttr1());  
    this->setAttr2(instanceOfCName.getAttr2());  
}
```

Association entre classes

Propriétés de l'extrémité de l'association

- type
- visibility
- ownerScope
- multiplicity
- ordering
- changeability
- aggregation

Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..1 (p. ex. visibilité public)

```
class CName1 {
```

```
// — Implémentation cachée de l'objet en association
```

```
private CName2 roleName;
```

```
// — Access public à l'objet en association
```

```
public CName2 getRoleName() {
```

```
    return this.roleName;
```

```
}
```

```
// — Modification possible de la référence de l'objet en association
```

```
public void setRoleName(CName2 instance) {
```

```
    if (!this.getRoleName().equals(instance))
```

```
        this.roleName = instance;
```

```
}
```

```
}
```

Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..1 (p. ex. visibilité public)

```
class CName1 {
```

```
// — Implémentation cachée de l'objet en association
```

```
    private CName2 roleName;
```

```
// — Access public à l'objet en association
```

```
    public CName2 getRoleName() {
```

```
        return this.roleName;
```

```
    }
```

```
// — Modification possible de la référence de l'objet en association
```

```
    public void setRoleName(CName2 instance) {
```

```
        if (!this.getRoleName().equals(instance))
```

```
            this.roleName = instance;
```

```
    }
```

```
}
```


Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..* ou 1..* (p. ex. visibilité public)

```
class CName1 {  
    // — Implémentation cachée de la collection (= ensemble)  
    private Set<CName2> roleName = new HashSet<CName2>();  
  
    // — Access public à l'ensemble d'objets en association  
    public Set<CName2> getRoleName() {  
        return new Set<CName2>(roleName);  
    }  
    public void setRoleName(Set<CName2> setOfCName2) { ... }  
  
    // — Modification de l'ensemble d'objets en association  
    public boolean addToRoleName(CName2 instance) { ... }  
    public boolean removeFromRoleName(CName2 instance) { ... }  
  
    // — Informations sur l'ensemble d'objets en association  
    public boolean hasInRoleName(CName2 instance) { ... }  
    public int cardOfRoleName() { ... }  
}
```

Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..* ou 1..* (p. ex. visibilité public)

```
class CName1 {  
    // — Implémentation cachée de la collection (= ensemble)  
    private Set<CName2> roleName = new HashSet<CName2>();  
  
    // — Access public à l'ensemble d'objets en association  
    public Set<CName2> getRoleName() {  
        return new Set<CName2>(roleName);  
    }  
    public void setRoleName(Set<CName2> setOfCName2) { ... }  
  
    // — Modification de l'ensemble d'objets en association  
    public boolean addToRoleName(CName2 instance) { ... }  
    public boolean removeFromRoleName(CName2 instance) { ... }  
  
    // — Informations sur l'ensemble d'objets en association  
    public boolean hasInRoleName(CName2 instance) { ... }  
    public int cardOfRoleName() { ... }  
}
```

Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..* ou 1..* (p. ex. visibilité public)

```
class CName1 {  
    // — Implémentation cachée de la collection (= ensemble)  
    private Set<CName2> roleName = new HashSet<CName2>();  
  
    // — Access public à l'ensemble d'objets en association  
    public Set<CName2> getRoleName() {  
        return new Set<CName2>(roleName);  
    }  
    public void setRoleName(Set<CName2> setOfCName2) { ... }  
  
    // — Modification de l'ensemble d'objets en association  
    public boolean addToRoleName(CName2 instance) { ... }  
    public boolean removeFromRoleName(CName2 instance) { ... }  
  
    // — Informations sur l'ensemble d'objets en association  
    public boolean hasInRoleName(CName2 instance) { ... }  
    public int cardOfRoleName() { ... }  
}
```

Association unidirectionnelle en Java

Exemple de CName1 vers CName2

- Multiplicity = 0..* ou 1..* (p. ex. visibilité public)

```
class CName1 {  
    // — Implémentation cachée de la collection (= ensemble)  
    private Set<CName2> roleName = new HashSet<CName2>();  
  
    // — Access public à l'ensemble d'objets en association  
    public Set<CName2> getRoleName() {  
        return new Set<CName2>(roleName);  
    }  
    public void setRoleName(Set<CName2> setOfCName2) { ... }  
  
    // — Modification de l'ensemble d'objets en association  
    public boolean addToRoleName(CName2 instance) { ... }  
    public boolean removeFromRoleName(CName2 instance) { ... }  
  
    // — Informations sur l'ensemble d'objets en association  
    public boolean hasInRoleName(CName2 instance) { ... }  
    public int cardOfRoleName() { ... }  
}
```

Association unidirectionnelle en C++

Exemple de CName1 vers CName2

- Multiplicity = 0..1

```
class CName1 {  
public :  
    inline virtual const CName2 * getRoleName(void) const ;  
    inline virtual void setRoleName(CName2 * instance) ;  
private :  
    CName2 * _roleName ;  
};  
  
inline const CName2 * CName1 : :getRoleName(void) const {  
    return _roleName ;  
}  
  
inline void CName1 : :setRoleName(CName2 * instance) {  
    if (getRoleName() != instance) {  
        _roleName = instance ;    _invariant() ;  
    }  
}
```

Association unidirectionnelle en C++

Exemple de CName1 vers CName2

- Multiplicity = 0..1

```
class CName1 {  
public :  
    inline virtual const CName2 * getRoleName(void) const ;  
    inline virtual void setRoleName(CName2 * instance);  
private :  
    CName2 * _roleName;  
};  
inline const CName2 * CName1 : :getRoleName(void) const {  
    return _roleName;  
}  
inline void CName1 : :setRoleName(CName2 * instance) {  
    if (getRoleName() != instance) {  
        _roleName = instance;    _invariant();  
    }  
}
```

Association unidirectionnelle en C++

Exemple de CName1 vers CName2

- Multiplicity = 0..1

```
class CName1 {  
public :  
    inline virtual const CName2 * getRoleName(void) const ;  
    inline virtual void setRoleName(CName2 * instance) ;  
private :  
    CName2 * _roleName ;  
};  
inline const CName2 * CName1 : :getRoleName(void) const {  
    return _roleName ;  
}  
inline void CName1 : :setRoleName(CName2 * instance) {  
    if (getRoleName() != instance) {  
        _roleName = instance ;    _invariant() ;  
    }  
}
```

Association unidirectionnelle en C++

Multiplicity = 0..* ou 1..*

```
class CName1 {
public :

// — Global access to the set of instances 'roleName'
inline virtual const set < CName2 * > & getRoleName(void) const ;
inline virtual void setRoleName(const set < CName2 * > & setOfInstance) ;

// — Add/remove one instance in/from 'roleName'
inline virtual bool addToRoleName(CName2 & instance) ;
inline virtual bool removeFromRoleName(CName2 & instance) ;

// — Get information about the set 'roleName'
inline virtual bool hasInRoleName(const CName2 & instance) const ;
inline virtual unsigned int cardOfRoleName(void) const ;

// — Hidden implementation of 'roleName'
private :
    set < CName2 * > _roleName ;
};
```


Association unidirectionnelle en C++

Multiplicity = 0..* ou 1..*

```
class CName1 {  
    public :  
  
    // — Global access to the set of instances 'roleName'  
    inline virtual const set < CName2 * > & getRoleName(void) const ;  
    inline virtual void setRoleName(const set < CName2 * > & setOfInstance) ;  
  
    // — Add/remove one instance in/from 'roleName'  
    inline virtual bool addToRoleName(CName2 & instance) ;  
    inline virtual bool removeFromRoleName(CName2 & instance) ;  
  
    // — Get information about the set 'roleName'  
    inline virtual bool hasInRoleName(const CName2 & instance) const ;  
    inline virtual unsigned int cardOfRoleName(void) const ;  
  
    // — Hidden implementation of 'roleName'  
    private :  
        set < CName2 * > _roleName ;  
};
```

Association unidirectionnelle en C++

Multiplicity = 0..* ou 1..*

```
class CName1 {  
    public :  
  
    // — Global access to the set of instances 'roleName'  
    inline virtual const set < CName2 * > & getRoleName(void) const ;  
    inline virtual void setRoleName(const set < CName2 * > & setOfInstance) ;  
  
    // — Add/remove one instance in/from 'roleName'  
    inline virtual bool addToRoleName(CName2 & instance) ;  
    inline virtual bool removeFromRoleName(CName2 & instance) ;  
  
    // — Get information about the set 'roleName'  
    inline virtual bool hasInRoleName(const CName2 & instance) const ;  
    inline virtual unsigned int cardOfRoleName(void) const ;  
  
    // — Hidden implementation of 'roleName'  
    private :  
        set < CName2 * > _roleName ;  
};
```

Association unidirectionnelle en C++

Multiplicity = 0..* ou 1..*

```
class CName1 {  
    public :  
  
    // — Global access to the set of instances 'roleName'  
    inline virtual const set < CName2 * > & getRoleName(void) const ;  
    inline virtual void setRoleName(const set < CName2 * > & setOfInstance) ;  
  
    // — Add/remove one instance in/from 'roleName'  
    inline virtual bool addToRoleName(CName2 & instance) ;  
    inline virtual bool removeFromRoleName(CName2 & instance) ;  
  
    // — Get information about the set 'roleName'  
    inline virtual bool hasInRoleName(const CName2 & instance) const ;  
    inline virtual unsigned int cardOfRoleName(void) const ;  
  
    // — Hidden implementation of 'roleName'  
    private :  
        set < CName2 * > _roleName ;  
};
```

Association bidirectionnelle en Java

Gestion de l'intégrité du référencement réciproque

exemple : Animal – Place

```
private Place location;  
  
public Place getLocation() { return location; }  
  
public void setLocation(Place location) {  
    if (location != null)  
        if (this.location != null && this.location != location)  
            this.location.removeFromLocalPopulation(this);  
    if (this.location != location) {  
        this.location = location;  
        location.addToLocalPopulation(this);  
    }  
}
```

Association bidirectionnelle en Java

Gestion de l'intégrité du référencement réciproque

exemple : Animal – Place

```
class Place {  
  
    private Set<Animal> localPopulation = new HashSet<Animal>();  
  
    public void addToLocalPopulation(Animal animal) {  
        if (!this.localPopulation.contains(animal)) {  
            this.localPopulation.add(animal);  
            animal.setLocation(this);  
        }  
    }  
}
```

Classe–Association en Java

[1/4]

Exemple de la relation prédateurs – proies (classe Species)

```
class Species {  
    private Set<FoodLink> preys = new HashSet<FoodLink>();  
    ...  
    private Set<FoodLink> predators = new HashSet<FoodLink>();  
    ...  
}
```

```
class FoodLink {  
    private Species prey;  
    private Species predator;  
    private double preference;  
    ...  
}
```

Classe–Association en Java

[2/4]

Exemple de la relation prédateurs – proies (classe Species)

```
public boolean hasInPreys(Species prey) {  
    FoodLink fl = new FoodLink(this, prey, 0);  
    Iterator<FoodLink> iter = preys.iterator();  
    boolean found = false;  
    while (iter.hasNext() && !found) found = iter.next().equals(fl);  
    return found;  
}
```

```
public boolean addInPreys(Species prey, double preference) {  
    // check that the species is not already referenced as a prey  
    FoodLink fl = new FoodLink(this, prey, preference);  
    Iterator<FoodLink> iter = preys.iterator();  
    boolean found = false, added = false;  
    while (iter.hasNext() && !found) found = iter.next().equals(fl);  
    if (!found) added = preys.add(fl) && prey.predators.add(fl);  
    return added;  
}
```

Classe–Association en Java

[3/4]

Exemple de la relation prédateurs – proies (classe Species)

```
public boolean hasInPredators(Species predator) {  
    FoodLink fl = new FoodLink(predator, this, 0);  
    Iterator<FoodLink> iter = predators.iterator();  
    boolean found = false;  
    while (iter.hasNext() && !found)  
        found = iter.next().equals(fl);  
    return found;  
}
```


Classe–Association en Java

[4/4]

Exemple de la relation prédateurs – proies (classe FoodLink)

```
public class FoodLink {  
    public FoodLink(Species predator, Species prey) {  
        this.prey = prey;  
        this.predator = predator;  
    }  
  
    public FoodLink(Species predator, Species prey, double preference) {  
        this.prey = prey;  
        this.predator = predator;  
        this.preference = preference;  
    }  
  
    public boolean equals(Object o) {  
        if (!(o instanceof FoodLink)) return false;  
        FoodLink f = (FoodLink)o;  
        return f.prey == this.prey && f.predator == this.predator;  
    }  
}
```

Association qualifiée en Java

Exemple : Savanna – Species

```
private static Map<String, Species> species = new HashMap<String, Species>();
```

```
public static Species getSpecies(String name) {  
    return Savanna.species.get(name);  
}
```

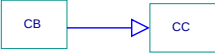
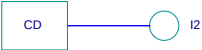
```
public static void addToSpecies(String name, Species species) {  
    Savanna.species.put(name, species);  
}
```

```
public static void removeFromSpecies(String name) {  
    Savanna.species.remove(name);  
}
```

```
public static boolean hasInSpecies(String name) {  
    return Savanna.species.containsKey(name);  
}
```

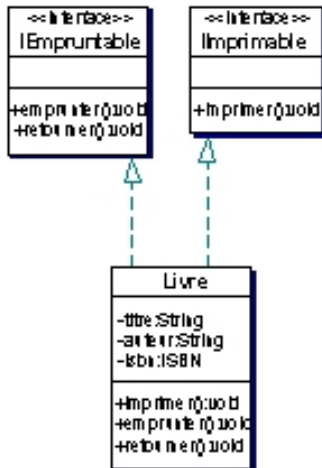
```
public static int numberOfSpecies() { return Savanna.species.size();}
```

Classes, interface, généralisation, implémentation

	java	<code>public class CA { /* */ }</code>
	C++	<code>class CA { /* */ };</code>
	java	<code>public abstract class CA { /* */ }</code>
	C++	<code>class CB { /* */ };</code>
	java	<code>public interface I1 { /* */ }</code>
	C++	<code>class I1 { /* */ };</code>
	java	<code>public class CB extends CC { /* */ }</code>
	C++	<code>class CB: public CC { /* */ };</code>
	java	<code>public class CD implements I2 { /* */ }</code>
	C++	<code>class CD: public I2 { /* */ };</code>

Réalisation en Java

UML



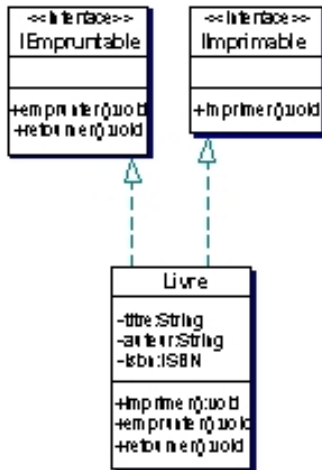
Java

```

public class Livre implements
    IImprimable, IEmpruntable {
    private String titre;
    private String auteur;
    private ISBN isbn;
    public void Imprimer(){
        ...
    }
    public void Emprunter(){
        ...
    }
    public void Retourner(){
        ...
    }
}
  
```

Réalisation en C++

UML



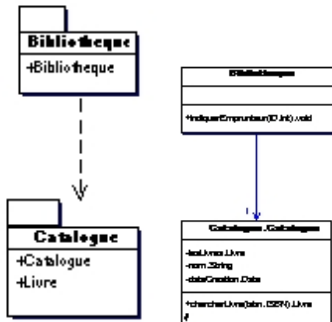
C++

```

public class Livre :
    IImprimable, IEmpruntable {
private string titre;
private string auteur;
private ISBN isbn;
public void Imprimer(){
    ...
}
public void Emprunter(){
    ...
}
public void Retourner(){
    ...
}
}
  
```

Dépendance

UML



Java

```

package Bibliotheque;

import catalogue;

public class Bibliotheque {
    private Catalogue leCatalogue;
    ...
}
  
```

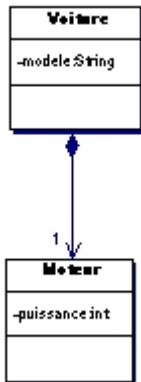
C++

```

namespace Bibliotheque {
    using Catalogue;
    public class Bibliotheque {
        private Catalogue leCatalogue;
        ...
    }
}
  
```

Agrégation

UML



Java

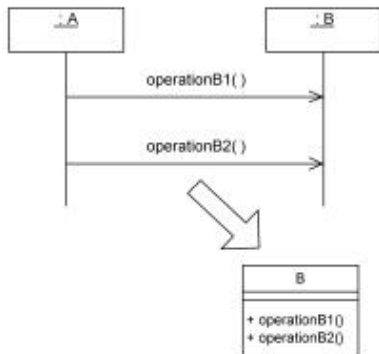
```
public class Voiture {
    private String modele;
    private Moteur moteur;
    private static class Moteur {
        private int puissance;
    }
    ... }
```

C++

```
public class Voiture {
    private string modele;
    private Moteur moteur;
    private class Moteur {
        private int puissance;
    }
    ... }
```

Séquence

UML

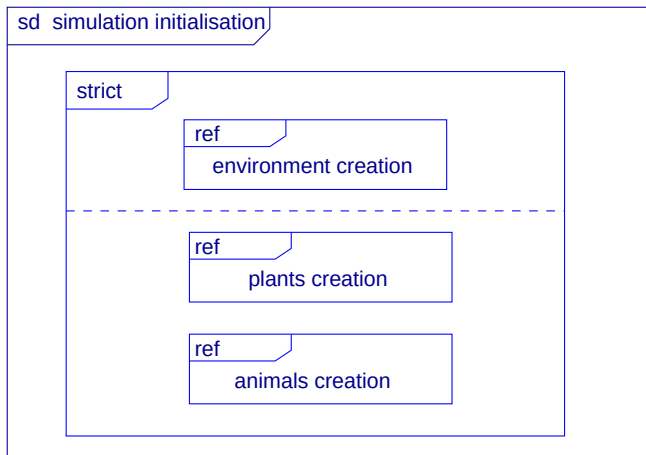


C++

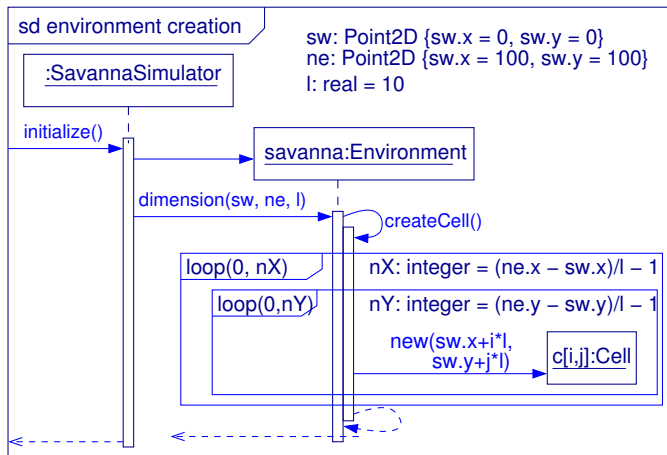
```
public class A {
    ...
    ... leB.OperationB1();
    ... leB.OperationB2();
    ...
}

public class B {
    ...
    public void OperationB1();
    public void OperationB2();
    ...
}
```


Exemple interactions : initialisation de la simulation



Exemple interactions : création de l'environnement



Exemple interactions : création des cellules

```
public void dimension(Point2D sw, Point2D ne, double l) {  
    assert(l > 0);  
    assert(ne.getX() > sw.getX() && ne.getY() > sw.getY());  
    southWestPoint = sw;  
    northEastPoint = ne;  
    setCellSize(l);  
    createCells();  
}
```

Exemple interactions : création des cellules

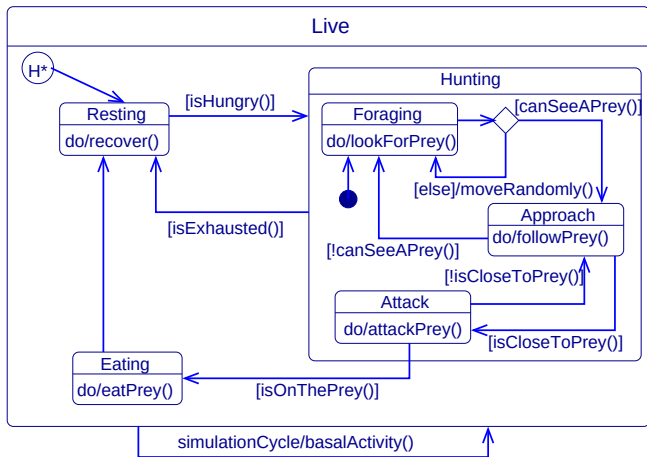
```
protected void createCells() {  
    if (getCellSize() > 0) {  
        float l = getCellSize();  
        int nX = (int)((getXMax() - getXMin()) / l) - 1;  
        int nY = (int)((getYMax() - getYMin()) / l) - 1;  
        cells = new Cell[nX][nY];  
  
        for (int i = 0; i <= nX; i++)  
            for (int j = 0; j <= nY; j++)  
                cells[i][j] = new Cell(getXMin() + i * l,  
                                         getYMin() + j * l);  
    }  
}
```

Modèle État–Transition : sémantique opérationnelle

Traduction d'un modèle état-transition

- **État** : observable défini par un ensemble de propriétés d'un classifier, la réalisation d'une activité ou une situation d'attente
- **Transition** : changement d'état possible
- **Événement** : occurrence pouvant provoquer le déclenchement d'une transition
- Sémantique (machine à état *comportementale*)
 - Enchaînements d'activités
 - Structures de contrôle (séq. ou par.)
 - Machine à états : dépendant de la plate-forme
 - Restrictions → implémentations particulières

Exemple Etat-Transition : comportement d'un Lion



Exemple Etat-Transition : comportement d'un Lion

propriétés de ce modèle

- Pas d'événement sur les transitions « internes »
- Transitions gardées (éventuellement toujours vraie)
- Déterministe
- Pas de régions orthogonales
- Actions associées aux états (et aux transitions)

Exemple Etat-Transition : comportement d'un Lion

remarques sur ce modèle

- Toutes les actions dont la réalisation est contrôlée par la machine à état sont des opérations **protected**
- la gestion de l'état composite **Hunting** est assurée par une méthode dédiée.

Motivations :

- Homogénéité de la transformation
- **Spécialisation comportementale** : cohérence avec le modèle de classe
- Les conditions définissant les gardes des transitions sont assurées par des opérations **protected** {isQuery}

Modèle d'activités

Traduction du modèle en code

- **Flots de contrôle** : parallélisme ?
 - multi-threads ?
 - multi-tâches ?
 - répartis sur plusieurs unités indépendantes ?
- **Flots de données** : support de communication ?
 - données partagées ?
 - réseaux de communication ?
- **Sémantique**
 - Réalisation d'activités (actions)
 - Structures de contrôle
 - Communication (asynchrone)
 - Gestion des exceptions

Modèle d'activités

Traduction du modèle en code

- **Flots de contrôle** : parallélisme ?
 - multi-threads ?
 - multi-tâches ?
 - répartis sur plusieurs unités indépendantes ?
- **Flots de données** : support de communication ?
 - données partagées ?
 - réseaux de communication ?
- **Sémantique**
 - Réalisation d'activités (actions)
 - Structures de contrôle
 - Communication (asynchrone)
 - Gestion des exceptions

Modèle d'activités

Traduction du modèle en code

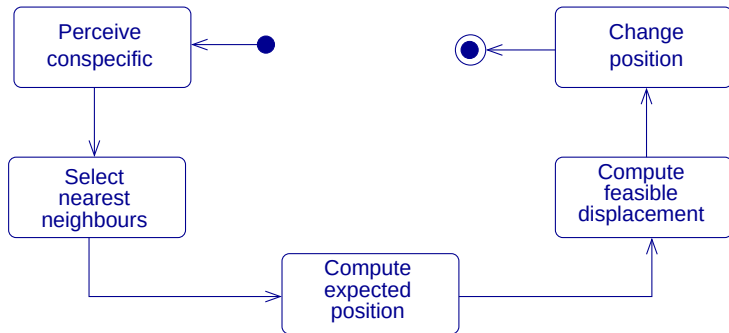
- **Flots de contrôle** : parallélisme ?
 - multi-threads ?
 - multi-tâches ?
 - répartis sur plusieurs unités indépendantes ?
- **Flots de données** : support de communication ?
 - données partagées ?
 - réseaux de communication ?
- **Sémantique**
 - Réalisation d'activités (actions)
 - Structures de contrôle
 - Communication (asynchrone)
 - Gestion des exceptions

Typologie des actions

- Création – destruction d'objet
- Lecture – Modification de la valeur d'un attribut
- Mise en relation d'objets | suppression de la relation
- Appel – exécution d'une opération
- Exécution d'une fonction modifiant la valeur d'une variable
- Communication : émission – réception de message

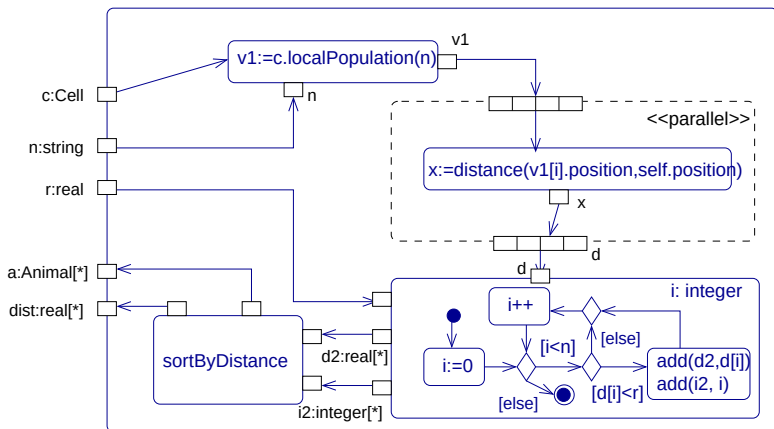
Vie des animaux : comportement de socialisation

Exemple de modèle d'activités



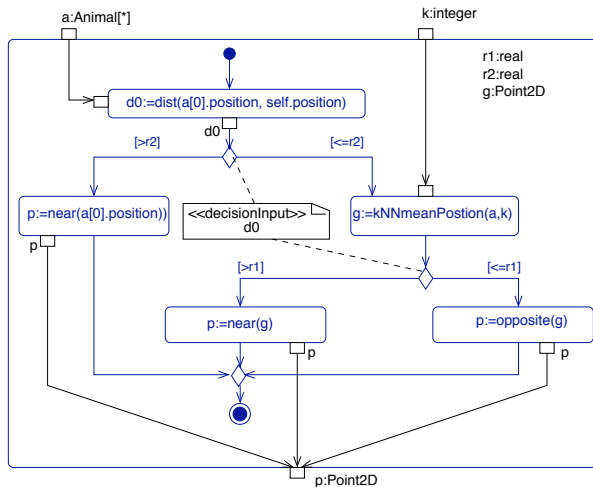
Vie des animaux : perception des congénères

Exemple de modèle d'activités



Vie des animaux : positionnement / congénères

Exemple de modèle d'activités



Modèle d'activités : implémentation

● Avantages

- Sémantique implémentable sur de nombreuses plates-formes
- Spécification précise (détaillée) des algorithmes
- Génération de test
- Vérification possible de propriétés du modèle (sans exécution)

● Inconvénients

- Lourdeur de l'écriture
- Norme encore peu précise
- Absence d'un langage (*unifié*) d'action

Modèle d'activités : implémentation

• Avantages

- Sémantique implémentable sur de nombreuses plates-formes
- Spécification précise (détaillée) des algorithmes
- Génération de test
- Vérification possible de propriétés du modèle (sans exécution)

• Inconvénients

- Lourdeur de l'écriture
- Norme encore peu précise
- Absence d'un langage (*unifié*) d'action

- 4 Le modèle états-transitions : concepts de base
- 5 Le modèle états-transitions : concepts avancés
- 6 Le modèle des activités : processus métier
- 7 Le modèle des activités : du modèle au programme

- 4 Le modèle états-transitions : concepts de base
- 5 Le modèle états-transitions : concepts avancés
- 6 Le modèle des activités : processus métier
- 7 Le modèle des activités : du modèle au programme

- 4 Le modèle états-transitions : concepts de base
- 5 Le modèle états-transitions : concepts avancés
- 6 Le modèle des activités : processus métier
- 7 Le modèle des activités : du modèle au programme

- 4 Le modèle états-transitions : concepts de base
- 5 Le modèle états-transitions : concepts avancés
- 6 Le modèle des activités : processus métier
- 7 Le modèle des activités : du modèle au programme